Understanding the Energy Use of Multimodal AI Inference Across the Compute Continuum: Current Trends and Future Scenarios

J.J. Hernández-Carlón*, A. Mauro†, A.M. Tulino†, J. Llorca*
*Centre Tecnològic de Telecomunicacions de Catalunya (CTTC), Spain
†University of Naples Federico II (UNINA), Italy

***Abstract*—AI inference can be executed in hyperscale core data centers, metropolitan edge sites, or compute-enabled access nodes, and the choice has a first-order impact on energy: inner tiers operate more efficient facilities, but reaching them requires hauling data across the network. We study this trade-off through a Cloud Network Flow model that represents the computation, memory, and communication requirements of multimodal AI inference as flows over a multidimensional infrastructure graph, charges every resource — accelerators, router ports, and optical channels — with an idle-plus-load-proportional power model that includes host, over-provisioning, and facility overheads, and solves energy-optimal placement, routing, and resource allocation as a mixed-integer linear program subject to end-to-end latency constraints. We show in closed form that optimal placement is driven by two workload descriptors — the computation-to-communication ratio and the computation-to-memory ratio — together with infrastructure descriptors that capture the energy efficiency of each resource across tiers. We validate the resulting placement rule against the MILP optimum across representative generative and analytics services, and quantify how efficiency gains, accelerator memory and throughput scaling, facility-efficiency evolution, and tightening latency budgets reshape placement and total power across the continuum.**

***Index Terms*—AI inference, multimodal AI, energy efficiency, edge computing, service placement, cloud network flow, PUE.**

I. Introduction

The energy demand of AI is increasingly driven by inference, as models are served continuously to large user and device populations [1]. The infrastructure that can host inference is no longer confined to hyperscale data centers: cloud providers deploy accelerators in metropolitan zones [2], and operators bring compute to aggregation and radio sites through multi-access edge computing [3]. Next-generation networks are rapidly evolving into tightly integrated compute-communication systems, forming what is commonly referred to as the *distributed compute continuum*: communication, computing, and storage resources spanning centralized/core data centers, edge sites, access points, and end user devices [4]. Where a service should run along this core–edge–access continuum is therefore a practical decision with a direct energy impact.

More centralized tiers, such as core data centers, typically benefit from economies of scale and lower power usage effectiveness (PUE) [5], potentially reducing the energy per delivered FLOP. However, serving requests at these locations may require transporting data across more network segments, over communication hardware that is far from energy-proportional and provisioned well above its carried load. Modern AI services, in turn, exhibit heterogeneous resource requirements depending on their inference paradigm, backbone architecture, and input/output modalities, which determine their computational workload, memory footprint, and request/response communication demands. Their energy-efficient placement is thus inherently workload- and infrastructure-dependent.

This paper develops a cloud network flow (CNFlow) based model that accounts for the computation, memory, and communication requirements of AI inference together with tier-dependent energy costs and end-to-end latency constraints, enabling the study of energy-optimal deployments for a wide range of resource-intensive multimodal AI services. Our contributions are:

- *Infrastructure and service model.* We specialize the CNFlow framework [6], [7] to AI inference over a core–edge–access hierarchy, with a unified energy model in which every resource — accelerator, router port, optical channel — draws idle plus load-proportional power, scaled by host overhead, over-provisioning, and facility PUE (Sec. IV), and an analytical service model that derives per-request computation, memory, and communication demands from model architecture and workload characteristics across autoregressive, diffusion, and discriminative paradigms (Sec. V).
- *Formulation.* We formulate energy-optimal placement, routing, and resource allocation as a MILP with integer accelerator and channel counts, subject to capacity and end-to-end latency constraints (Sec. VI).
- *Threshold-based placement rule.* We prove that energy-optimal placement can be characterized by two infrastructure-dependent thresholds on the computation-to-communication ratio I, with the computation-to-memory ratio σ bending both thresholds for memory-bound services (Sec. VII).
- *Evaluation.* Across representative generative and analytics services, we validate the placement rule against the MILP optimum and quantify the impact of service nature, infrastructure efficiency and capacity evolution, facility-efficiency (PUE) scenarios, and tightening latency budgets on placement and total power (Sec. VIII).

II. Related Work

Service placement as network flow. Service distribution in distributed cloud networks [8], [9] and virtual network function placement [10] jointly decide where functions execute and how traffic is routed, for generic functions with fixed per-unit costs; CNFlow [6], [7] unifies communication, computation, and storage as flows over one graph, and we specialize its resource and cost model to AI inference. *Energy of AI inference.* Serving systems and measurement studies characterize accelerator power, idle draw, and power management for language models inside data centers [11]–[13], without addressing where across a network hierarchy inference should run. Recent edge–cloud LLM serving frameworks schedule or split inference between edge and cloud resources to reduce energy and latency [14]–[16], but treat the network as a fixed cost between two tiers; in contrast, we model the network itself — its segments, ports, and optics — as a first-class energy consumer, and derive placement conditions in closed form across a multi-tier continuum. *Processing versus transport energy.* For conventional cloud services, transport energy can dominate processing energy [17], [18], and power models of IP-over-WDM networks quantify the per-bit cost of routers and optics [19]–[21]; we combine these strands into a placement rule parameterized by the nature of AI services.

III. Cloud Network Flow Overview

This section introduces the formal modeling framework adopted in this work. We build upon the Cloud Network Flow (CNFlow) paradigm introduced in [8], [9] and generalized in [6], [7], which provides a unified graph-based abstraction for jointly representing communication, computation, and storage resources within modern compute-integrated networks. Unlike traditional approaches that treat computation and communication as separate modeling and/or optimization domains, CNFlow encodes all resource types as edges of a single directed graph, reducing the problem of end-to-end orchestration across a generic distributed/networked computing infrastructure to a generalized network flow problem [6]–[9].

A. Cloud Network Model

In the CNFlow framework, a completely general network infrastructure is modeled as a directed graph over which computation and communication resources are jointly represented.

1) Cloud-Network Graph: In line with [6]–[9], we model a generic cloud-integrated network as a directed (cloud-augmented) graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where in addition to communication links representing the capability to transmit information between communication nodes, $\mathcal{E}$ includes production, consumption, and computation links representing the capability to produce, consume, and process information at a particular network location, as illustrated in Fig. 1.

As such, a node in the cloud network graph $\mathcal{G}$ can either be a communication node ($\mathcal{V}^c$), a production node ($\mathcal{V}^s$), a consumption node ($\mathcal{V}^d$), or a computation node ($\mathcal{V}^p$). Furthermore, $\mathcal{E}^c$, $\mathcal{E}^p$, $\mathcal{E}^s$, and $\mathcal{E}^d$ represent the set of communication, computation, production, and consumption links, respectively. Hence, $\mathcal{V} = \mathcal{V}^c \cup \mathcal{V}^p \cup \mathcal{V}^s \cup \mathcal{V}^d$ and $\mathcal{E} = \mathcal{E}^c \cup \mathcal{E}^p \cup \mathcal{E}^s \cup \mathcal{E}^d$.

2) Cloud-Network Graph Attributes: Each link $(u, v) \in \mathcal{E}$ in the cloud-network graph represents a (possibly multi-dimensional) production, consumption, communication, or computation resource, characterized by two fundamental parameters:

- **Capacity** c_{uv}: the maximum throughput of the resource, expressed in the native flow units of its resource type. For *communication links* $(u, v) \in \mathcal{E}$, capacity may be measured in e.g., bits per second (bps). For *computation links* $(u, v) \in \mathcal{E}^p$, capacity may be measured in e.g., floating-point operations per second (FLOPs) and/or memory/storage units, e.g., bytes.
- **Unit cost** w_{uv}: the operational cost of the corresponding resource. This cost parameter may reflect the economic expenditure associated with operating a given resource over a given period, such as the hourly cost per Mbps of operating a communication link or the hourly cost per GFLOPs of operating a computation resource. It may also represent the power consumption incurred while operating the resource, expressed in e.g., Watt per bps (equivalently, Joule/bit) or Watt per FLOPs (equivalently, Joule/FLOP).

Source and destination links ($\mathcal{E}^s$, $\mathcal{E}^d$) act as logical ingress and egress points for service data streams and are assumed to carry zero cost with sufficiently large capacity.

We denote by $\mathcal{V}^-(u)$ and $\mathcal{V}^+(u)$ the set of incoming and outgoing neighbors of node $u \in \mathcal{V}$, respectively.

This multi-dimensional resource representation constitutes a key advantage of the CNFlow framework: by encoding communication, computation, and storage resources as edges within a unified graph, the orchestration problem across the distributed compute continuum can be addressed through a single flow optimization formulation on the cloud-network graph $\mathcal{G}$.

B. Service Model

Following the service graph model introduced in [6], [7], [9], each service is represented as a directed acyclic graph (DAG) that captures both the functional dependencies among processing stages and the associated computation and communication resource requirements. This representation naturally accommodates the multi-stage structure of AI inference pipelines, where inputs, intermediate computations, and outputs impose heterogeneous demands on communication, processing, and memory resources across the infrastructure. The following subsections formally define the service DAG structure and the associated multi-dimensional resource requirements.

1) Service Graph: A generic service, or a collection of related services, is represented as a directed acyclic graph (DAG) $\mathcal{R} = (\mathcal{I}, \mathcal{K})$, where vertices $\mathcal{I}$ represent *service functions* (i.e., individual processing stages or microservices) and edges $\mathcal{K}$ represent *data streams*, also referred to as *commodities*, flowing between functions. An edge $k \equiv (i, j) \in \mathcal{K}$ denotes a data stream produced by function i and consumed by function j. For each commodity k, we use $\mathcal{X}(k)$ to denote

the set of input commodities required to produce k, capturing the functional dependency structure of the service.

The vertices of the service graph are classified into three categories:

- **Source functions** $\mathcal{I}^s$: vertices with no incoming edges, representing data-producing endpoints (e.g., sensor feeds, user query ingress points). Each source function $i \in \mathcal{I}^s$ is associated with a fixed location $s^{\mathcal{I}}(i) \in V^s$ in the cloud-network graph, corresponding to the physical location of the data source or network ingress point.
- **Destination functions** $\mathcal{I}^d$: vertices with no outgoing edges, representing data-consuming endpoints (e.g., user response delivery, display rendering). Each destination function $i \in \mathcal{I}^d$ is associated with a fixed location $d^{\mathcal{I}}(i) \in V^d$.
- **Processing functions** $\mathcal{I}^p$: all intermediate vertices whose placement across the cloud-network is subject to optimization. These functions perform the data transformation and computation stages of the service pipeline.

Correspondingly, the set of commodities $\mathcal{K}$ is partitioned into source commodities $\mathcal{K}^s$ (produced by source functions), destination commodities $\mathcal{K}^d$ (consumed by destination functions), and processing commodities $\mathcal{K}^p = \mathcal{K} \setminus \mathcal{K}^s$ (produced by processing functions). An illustrative example of a service DAG for an AI inference application is shown in Fig. 2.

2) Service Graph Attributes: A distinguishing feature of the CNFlow-based service model is that each commodity $k \in \mathcal{K}$ carries a *multi-dimensional rate requirement* R^k_{uv}, which specifies the resource demand imposed by commodity k when it traverses link $(u, v) \in E^a$. Since the cloud-augmented graph encodes heterogeneous resource types as different link categories, the rate of a given commodity varies depending on the type of resource it consumes:

- **Communication rate** (e.g., in Mbps): the bandwidth required when the commodity traverses a communication link $(u, v) \in E$.
- **Computation rate** (e.g., in GFLOPS): the processing demand when the commodity passes through a computation output link $(u, v) \in E^{p+}$.
- **Memory rate** (e.g., in MB): the memory footprint when the commodity passes through a computation input link $(u, v) \in E^{p-}$.

This multi-dimensional characterization captures the *flow scaling* nature of real-world services, where data streams change size and resource demands as they progress through successive processing stages. For instance, in an AI inference pipeline, a tokenized input prompt may impose modest communication bandwidth but substantial GPU FLOPS during the attention computation stage, while the generated output tokens require less computation but sustained memory bandwidth for KV-cache management.

IV. AI Network Infrastructure

We particularize the generic cloud-network model of Section III to a three-tier hierarchical architecture representative of modern distributed compute infrastructures. We first describe the network hierarchy, which maps onto the topology of the associated cloud-network graph $\mathcal{G}$, and then characterize the capacity and cost (particularized to power consumption) of its computation and communication resources, which map onto the attributes of the cloud-network graph $\mathcal{G}$.

A. Network Hierarchy and Site Distribution

The infrastructure is organized in three tiers, distinguished by geographic scope, resource availability, and facility efficiency. These dimensions are not independent: they are facets of a natural progression in site scale, which grows more granular toward the network edge, and which sets in turn how many sites a tier contains, how much capacity each holds, and how efficiently each converts drawn power into useful work.

- The **core tier** comprises hyperscale regions serving national or continental service areas: purpose-built, liquid-cooled facilities hosting the densest available accelerators and interconnect, today provisioned at the scale of tens to hundreds of megawatts [1].
- The **edge tier** comprises metropolitan data centers and converted central offices — regional points of presence, or cloud providers' local zones [2] — serving a city-scale service areas. These sites are typically air-cooled and host mid-range accelerators, with power envelopes today in the sub-megawatt to few-megawatt range.
- The **access tier** comprises aggregation sites — radio access network (RAN) hubs, pre-aggregation points, and the compute of multi-access edge computing (MEC) platforms [3] — each fronting tens of cell sites. Today provisioned at the smallest power envelopes in the hierarchy and constrained to compact air-cooled accelerators, these sites are being progressively rebuilt for virtualized RAN processing, which is what makes accelerator co-location there increasingly plausible.

Within the CNFlow framework, each site node $u \in \mathcal{V}^c$ is attached to one or more compute clusters in $\mathcal{V}^p$. We denote by $\mathcal{V}^p(u)$ the set of compute clusters at site u, and by $\mathcal{V}^{c,t} \subset \mathcal{V}^c$ the set of data center sites at tier $t \in \{\mathrm{core}, \mathrm{edge}, \mathrm{access}\}$. The hierarchical structure of the proposed network, including the spatial relationship between tiers and their interconnections, is illustrated in Fig. 1.

The efficiency of a site is measured by its *power usage effectiveness* (PUE), the ratio of total facility power to the power delivered to computing equipment; a PUE of 1.0 is ideal, and typical values fall between 1.1 and 2.0 [22], [23].

Three orderings follow from the progression in scale across this hierarchy.

- Site count grows outward, $|\mathcal{V}^{c,\mathrm{core}}| \leq |\mathcal{V}^{c,\mathrm{edge}}| \leq |\mathcal{V}^{c,\mathrm{access}}|$.
- Average site capacity falls outward, $\bar{C}^{\mathrm{core}} \geq \bar{C}^{\mathrm{edge}} \geq \bar{C}^{\mathrm{access}}$.
- Facility overhead rises outward, $\mathrm{PUE}^{\mathrm{core}} \leq \mathrm{PUE}^{\mathrm{edge}} \leq \mathrm{PUE}^{\mathrm{access}}$ [5], [23].

The last follows because cooling and power-distribution plant amortize better at scale, and because core sites may be sited for energy availability whereas outer sites must be sited for proximity to users; we quantify it through the facility efficiency factor $\mathrm{PEF}^t \triangleq \mathrm{PUE}^t / \mathrm{PUE}_{\mathrm{core}}$, with $\mathrm{PEF}^{\mathrm{core}} = 1$.

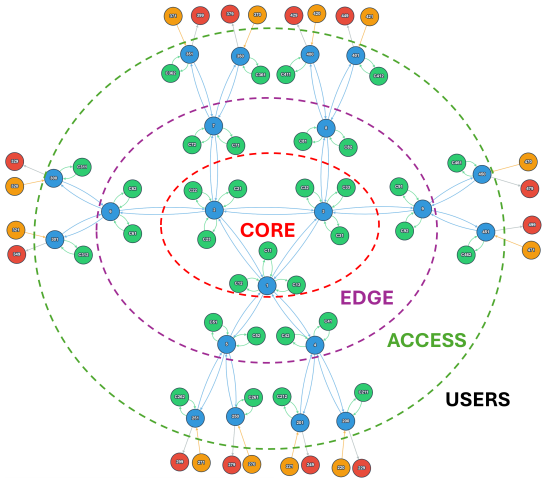


Fig. 1: Hierarchical network topology showing the spatial distribution of Core regions, Edge sites, and Access Points across the cloud-edge-access computing continuum

B. Energy Efficiency Coefficient

Computation and communication are provisioned in different physical artifacts but are consumed analogously: capacity is installed, held ready, and drawn upon.

We therefore model all resources with a unified relation. Each resource is described at three levels. The *elemental unit* is the smallest independently provisioned artifact — an accelerator, a router line card, an optical channel — characterized by a capacity C in the native flow units of its type, a peak power $P_{\max}$, and an idle power P_{idle}. The *aggregating box* is the shared assembly that must be energized for the unit to function — the host node around an accelerator, the router chassis around a line card, the amplified line system along a lightpath — and is represented by a multiplicative overhead $(1+\alpha)$. Provisioned capacity further exceeds carried load by a factor $\varphi \geq 1$, which collects the two forms of over-provisioning that operators design in deliberately: peak-to-mean traffic variation, φ_{traf}, and resilience, φ_{res}. The *facility* houses the box and imposes cooling and distribution overhead PUE.

An elemental unit's *average* power draw (over the considered time window or service session) is modeled as a linear interpolation between idle and peak draw, as a function of its *average* utilization u, the fraction of wall-clock time during which it is engaged [24]–[26]:

$$P(u) = P_{\text{idle}} + u\left(P_{\max} - P_{\text{idle}}\right), \tag{1}$$

The energy required to deliver one unit of work is then

$$e = \underbrace{\frac{P_{\text{idle}} + u\,(P_{\max} - P_{\text{idle}})}{\hat{c}\,u}}_{\text{elemental unit}} \cdot \underbrace{(1+\alpha)}_{\text{box}} \cdot \underbrace{\varphi}_{\text{provisioning}} \cdot \underbrace{\text{PUE}}_{\text{facility}} \tag{2}$$

where C is the element unit capacity.

Equation (2) is a critical relation in this paper. Read as a chain, it states the operating power of a unit, divides by what that unit actually delivers, and then charges the unit for the shared infrastructure it requires. It is an efficiency at a chosen operating point, determined by the utilization u, fixed, e.g., by quality of service (QoS) requirements (Section IV-C). Two remarks follow. First, the idle term does not vanish with load: as $u \to 0$, a unit amortizes its standing draw over vanishing output, which is why the operating utilization is a first-class parameter of the model. Second, (2) charges a single facility overhead PUE to the unit and its box, presuming both reside at one site; this holds for computation, but a communication link spans sites, and its facility overhead is composite (Section IV-D).

In the following, we characterize the capacity, power, and energy efficiency models for two main resource groups: computation and communication resources.

C. Computation Resources

We now describe the computation capacity and power models at each of the three aggregation levels, together with the resulting energy efficiency coefficient.

1) Elemental unit: the accelerator:

Capacity: An accelerator of type $a \in \mathcal{A}$ (e.g., GPU, TPU, NPU) is characterized by its peak compute throughput c_a^{flop}, in floating-point operations per second (FLOP/s or FLOPS) at the operating precision,[1] and its on-accelerator memory capacity c_a^{vram} in bytes, distinct from the host DRAM accounted for at the node/cluster level. Table I reports the ranges spanned by the accelerator classes considered. While the model applies to any accelerator class, in this work we focus on GPUs, the dominant platform for AI inference across all tiers of the continuum, and use the two terms interchangeably.

Power: Accelerator power draw follows the idle-plus-proportional model in (1) as a function of utilization $u_{a,m}^{\text{flop}}$, the fraction of wall-clock time an accelerator of type $a \in \mathcal{A}$ hosting model $m \in \mathcal{M}$ is engaged:

$$P_a(u_{a,m}^{\text{flop}}) = P_{\text{idle},a} + u_{a,m}^{\text{flop}}\left(P_{\text{TDP},a} - P_{\text{idle},a}\right), \tag{3}$$

where $P_{\text{TDP},a}$ is the thermal design power of accelerator a, the sustained draw at full load (peak draw), and $P_{\text{idle},a}$ its idle draw. The idle draw is that of an accelerator held ready to serve, and it is a fixed property of that ready state, independent of the workload resident on it [13]. It is therefore a single per-accelerator constant, which measurements place at 17–29% of TDP across typical data center GPUs [5], [28]. The utilization at which an accelerator operates, $u_{a,m}^{\text{flop}}$, depends on both the accelerator type and the hosted model,[2] and is capped by the QoS requirements of the corresponding service, as described in Section V.

2) Aggregating box: the compute node and cluster:

Capacity: Accelerators are aggregated into clusters of compute nodes. We assume a given compute cluster $p \in \mathcal{V}^p$ aggregates a total of G_p accelerators of a single type $a(p)$, with

$$C_p = G_p\, c_{a(p)}^{\text{flop}}, \qquad M_p = G_p\, c_{a(p)}^{\text{vram}}, \tag{4}$$

[1]Throughputs are quoted at dense 8-bit tensor precision. Vendors often report the twofold higher figure attainable under 2:4 structured sparsity [31], which production inference rarely exercises.

[2]In line with current practice, we assume a given accelerator hosts a single AI model, while different accelerators within a given cluster can host different models

TABLE I: Computation resource parameters. Accelerator ranges span typical data-center GPU classes (NVIDIA T4, L4, A10G, L40S, A100, H100, B200). Specific per-class values are used in Section VIII.

Symbol	Description	Typical range	Refs.
Elemental unit: the accelerator			
c_a^{flop}	peak compute throughput of accelerator type a, at dense 8-bit precision	0.13–4.5 PFLOP/s	[27]
c_a^{vram}	on-accelerator memory capacity	16–192 GB	[27]
$P_{\mathrm{TDP},a}$	thermal design power (peak sustained draw)	70–1000 W	[27]
$P_{\mathrm{idle},a}$	ready-to-serve idle draw	0.17–$0.29\,P_{\mathrm{TDP},a}$	[5], [13], [28]
$u_{a,m}^{\mathrm{flop}}$	utilization of an accelerator of type a hosting model m (QoS-capped; Sec. V)	0.6–0.8	—
Aggregating box: the compute node and cluster			
G_p	installed accelerators in cluster p	deployment-dependent	—
α_p	host-node overhead (CPU, DRAM, storage, I/O)	0.2–0.4	[11], [25]
y_p^m	number of active accelerators at cluster p serving model m	resource alloc. var. (Sec. VI)	—
Facility and provisioning			
PUE_u	facility overhead of site u (core / edge / access)	1.1–1.3 / 1.2–1.5 / 1.4–2.0	[5], [22], [23]
φ_u	compute over-provisioning (autoscaled pool)	1.0–1.3	[29], [30]
Energy efficiency coefficient			
$e_{u,p}^m$	energy per delivered FLOP, Eq. (8)	$\approx$ 0.4–0.9 pJ/FLOP	derived

denoting cluster p's compute capacity (peak compute throughput) and memory capacity, respectively.

Power: The accelerators cannot be used unless the host cluster nodes are energized: their CPU, DRAM, storage, and network interfaces draw power that performs no delivered work but scales with the number of accelerators served. We charge this as a multiplicative overhead $(1+\alpha_p)$ for cluster $p \in \mathcal{V}^p$, with $\alpha_p \approx 0.2$–0.4 for typical accelerator node configurations [11], [25]. Let $\mathcal{M}(p)$ denote the set of models hosted at cluster p, and y_p^m (with $\sum_{m\in\mathcal{M}(p)} y_p^m \le G_p$) the number of *active* accelerators at cluster p serving model m. The power consumed by cluster $p \in \mathcal{V}^p$ is then

$$P_p = \big(1+\alpha_p\big) \sum_{m\in\mathcal{M}(p)} y_p^m \, P_{a(p)}\big(u_{a(p),m}^{\mathrm{flop}}\big). \tag{5}$$

where we assume that all accelerators in a given cluster hosting a given model are uniformly load-balanced.

3) Facility: the data center site:

Capacity: A site $u \in \mathcal{V}^c$ brings its clusters under one administrative and infrastructural domain, and is the coarsest unit of computational resource within a tier, with

$$C_u = \sum_{p\in\mathcal{V}^p(u)} C_p, \qquad M_u = \sum_{p\in\mathcal{V}^p(u)} M_p, \tag{6}$$

denoting site u's compute capacity (peak compute throughput) and memory capacity, respectively.

Power: Cooling, power distribution, and auxiliary systems are charged through the site's PUE_u, and over-provisioning through φ_u, giving the total average power drawn at site $u \in \mathcal{V}^c$:

$$\boxed{P_u = \mathrm{PUE}_u\,\varphi_u \sum_{p\in\mathcal{V}^p(u)} \big(1+\alpha_p\big) \sum_{m\in\mathcal{M}(p)} y_p^m P_{a(p)}\big(u_{a(p),m}^{\mathrm{flop}}\big).} \tag{7}$$

We note that $\varphi_u \ge 1$ captures the power overhead due to additional units energized in reserve to absorb demand variations and failures. Thanks to the load balancing and horizontal autoscaling mechanisms standard in contemporary inference platforms [29], [30], which admit and retire replicas to keep the energized pool matched to demand, φ_u stays close to unity. Communication capacity, in contrast, is adapted at slower timescales, and its over-provisioning factors are correspondingly larger (see Section IV-D). Equation (7) is the compute term of the objective minimized in Section VI, evaluated at every site of $\mathcal{G}$.

4) Energy per delivered FLOP:

The energy efficiency (energy per unit of computation) of an accelerator of type a hosting model m in cluster p of site u, $e_{u,p}^m$, is then defined as

$$e_{u,p}^m = \frac{P_{a(p)}\big(u_{a(p),m}^{\mathrm{flop}}\big)}{c_{a(p)}^{\mathrm{flop}}\, u_{a(p),m}^{\mathrm{flop}}} \big(1+\alpha_p\big)\,\varphi_u\,\mathrm{PUE}_u \quad [\mathrm{J/FLOP}], \tag{8}$$

which serves as the specialization of (2) to computation, and together with its communication counterpart of Section IV-D constitutes the basis for the analysis of Section VII.

D. Communication Resources

Communication between sites is provisioned via two coupled sub-resources: *routing*, the packet processing performed at the routers terminating each link, and *transmission*, the transport of bits over optical channels along the fiber route. In line with modern IP-over-DWDM deployments, we assume the two are provisioned jointly: each provisioned channel occupies one router port and one coherent transceiver at each end of the link, rate-matched by construction [21], [32]. We describe them together at each aggregation level, mirroring the structure of Section IV-C.

TABLE II: Communication resource parameters, for the routing and transmission sub-resources.

Symbol	Description	Typical range	Refs.
Elemental units: the router port and the optical channel			
c_{uv}^{bw}	port/channel line rate (access–edge / edge–core)	100 / 400 Gb/s	[32], [33]
$P_{\max}^{\mathrm{port}}$	router port peak power (100G / 400G)	15–20 / 40–50 W	[20], [34]
$P_{\mathrm{idle}}^{\mathrm{port}}$	port idle draw	0.85–$0.95\,P_{\max}^{\mathrm{port}}$	[18], [26]
$P_{\max}^{\mathrm{ch}}$	coherent transceiver peak power (100G / 400G)	5–8 / 18–25 W	[21], [32]
$P_{\mathrm{idle}}^{\mathrm{ch}}$	transceiver idle draw	0.90–$0.95\,P_{\max}^{\mathrm{ch}}$	[21]
u_{uv}^{bw}	port/channel utilization (QoS-capped)	0.5–0.75	[19]
Aggregating boxes: the router chassis and the optical line system			
α_r	router chassis overhead (fabric, route processors, power conversion)	0.4–0.6	[19], [20], [35]
α_{ch}	line-system overhead per segment (amplifiers, ROADMs)	0.3–0.4	[17], [20]
n_{uv}	number of segments (IP-over-DWDM hops) per link (access–edge / edge–core)	1 / 2–4	[33], [36]
N_{uv}	number of available channels per link (single lit fiber pair)	64–96	[32]
y_{uv}	number of active channels on link (u, v)	resource alloc. variable (Sec. VI)	—
Facility and provisioning			
$\mathrm{PUE}_{uv}^{\mathrm{sites}}$	mean facility overhead along link (u, v)	1.15–1.75	derived
$\mathrm{PUE}_{uv}^{\mathrm{line}}$	line-site (hut/cabinet) facility overhead	1.5–2.0	[5], [17], [20]
φ_{uv}	over-provisioning factor, $\varphi_{uv}^{\mathrm{traf}} \cdot \varphi_{uv}^{\mathrm{res}}$	3–6	[19], [20], [37]
Link delay			
τ_{uv}	one-way propagation delay (access–edge / edge–core)	0.1–3 / 5–15 ms	[36], [38]
Energy efficiency coefficient			
e_{uv}	energy per delivered bit, Eq. (12)	$\approx$ 3–8 nJ/bit per segment	derived

1) Elemental units: the router port and the optical channel: Capacity: The elemental routing unit is a router port and the elemental transmission unit is an optical channel: a wavelength terminated by a coherent transceiver at each end, seated in the corresponding port. Port and channel share a common line rate c_{uv}^{bw}, in bits per second (bps), which is tier-dependent, ranging from lower-rate optics in the access and metro segments to high-rate coherent dense DWDM channels in the core [32].

Power: Both units follow the power model of (1) as a function of the link utilization u_{uv}^{bw}, the fraction of provisioned line rate carrying traffic; since port and channel carry the same bits and have the same capacity, a single utilization describes both. As on the compute side, its value is capped by QoS requirements, as queueing delay grows sharply with utilization. Per end of link segment, the port and transceiver draws are

$$P_{\mathrm{port}}(u_{uv}^{\mathrm{bw}}) = P_{\mathrm{idle}}^{\mathrm{port}} + u_{uv}^{\mathrm{bw}}\big(P_{\max}^{\mathrm{port}} - P_{\mathrm{idle}}^{\mathrm{port}}\big), \tag{9}$$

$$P_{\mathrm{ch}}(u_{uv}^{\mathrm{bw}}) = P_{\mathrm{idle}}^{\mathrm{ch}} + u_{uv}^{\mathrm{bw}}\big(P_{\max}^{\mathrm{ch}} - P_{\mathrm{idle}}^{\mathrm{ch}}\big). \tag{10}$$

Their idle fractions are higher than for computation: packet-processing silicon retains little dynamic range once energized, with idle draw measured at 85–95% of peak across router generations [18], [20], [26], while coherent transceivers run their DSP and forward-error-correction blocks at constant clock rates to maintain optical lock, making their draw almost load-agnostic, with idle fractions of 90–95% [21].

2) Aggregating boxes: the router chassis and the optical line system:

On the routing side, a port cannot operate without the line card that hosts it and the chassis around it: the switch fabric interconnecting line cards, route processors, and the chassis' internal power conversion must all be energized. As with the compute host node, we charge this as a multiplicative overhead $(1 + \alpha_r)$ on port power, with $\alpha_r \approx 0.4$–0.6 for typical router platforms [19], [20], [35].

On the transmission side, a channel cannot operate without the optical line system it rides on: inline amplifier sites along the fiber route and reconfigurable optical add-drop multiplexers at the link ends, shared by all channels provisioned on the same fiber. We charge its per-channel share as a multiplicative overhead $(1 + \alpha_{\mathrm{ch}})$ on transceiver power. For the reference topology of this work, $\alpha_{\mathrm{ch}} \approx 0.3$–$0.4$ for both link classes (access–edge and edge–core), dominated by the terminal equipment on short metro (access–edge) links and by the amplifier chain on long-reach (edge–core) ones [17], [20].

3) Facility: over-provisioning and energy overheads:

Capacity: We assume a given link $(u, v) \in \mathcal{E}^e$ aggregates a total of N_{uv} *end-to-end channels* (each comprising the lit wavelength and its terminating transceivers and ports), with $C_{uv} = N_{uv}\, c_{uv}^{\mathrm{bw}}$ denoting the resulting link capacity. Assuming a single lit fiber pair per segment, $N_{uv} \approx 64$–96 on standard DWDM grids [32].

Power: Power is drawn by the *active* end-to-end channels. Analogous to y_p^m on the compute side, we denote by $y_{uv} \leq$

N_{uv} the number of active channels on link (u, v). What differs from the compute side is the elasticity with which y_{uv} can follow demand. While compute autoscaling systems adapt y_p^m on short timescales, channels are provisioned through longer cycles. Software-defined networking (SDN) control is progressively narrowing this elasticity gap, but adaptation remains slower than compute autoscaling. This elasticity gap is charged on the power side: beyond the y_{uv} channels serving load, additional capacity is held energized to absorb traffic variations within the slow adaptation window and to protect against failures. We capture this through a single over-provisioning factor φ_{uv} per link: since in IP-over-DWDM both capacity headroom and protection are provisioned as additional end-to-end channels, the factor applies to the two sub-resources jointly. As introduced in Section IV-B, φ_{uv} comprises two components: peak-over-mean dimensioning, $\varphi_{uv}^{\mathrm{traf}} \approx$ 2–3, consistent with measured backbone utilizations [19], [37], and protection, $\varphi_{uv}^{\mathrm{res}} \approx 1.5$–2 [20], for combined values of $\varphi_{uv} = \varphi_{uv}^{\mathrm{traf}}\,\varphi_{uv}^{\mathrm{res}} \approx$ 3–6. The two sub-resources are further housed in different facility types, realizing the composite PUE overhead anticipated in Section IV-B: ports, chassis, and transceivers reside in the sites terminating each segment, charged the mean endpoint overhead $\mathrm{PUE}_{uv}^{\mathrm{sites}} = \frac{1}{2}(\mathrm{PUE}_u + \mathrm{PUE}_v)$.[3] The line system instead occupies small enclosures along the route with higher thermal overhead, $\mathrm{PUE}_{uv}^{\mathrm{line}} \approx 1.5$–$2.0$ [5], [17], [20]. The total average power drawn by link $(u, v) \in \mathcal{E}^c$ is then

$$P_{uv} \;=\; 2\,n_{uv}\,y_{uv}\,\varphi_{uv}\Big[\,P_{\mathrm{port}}\big(u_{uv}^{\mathrm{bw}}\big)\,\big(1+\alpha_r\big)\,\mathrm{PUE}_{uv}^{\mathrm{sites}} + P_{\mathrm{ch}}\big(u_{uv}^{\mathrm{bw}}\big)\,\big(\mathrm{PUE}_{uv}^{\mathrm{sites}} + \alpha_{\mathrm{ch}}\,\mathrm{PUE}_{uv}^{\mathrm{line}}\big)\Big] \tag{11}$$

where the factor $2\,n_{uv}$ counts the port–transceiver terminations at both ends of each of the link's n_{uv} segments (e.g., IP-over-DWDM hops).

4) Energy per delivered bit:

In line with Section IV-C4, we define the communication energy efficiency of link $(u, v) \in \mathcal{E}^c$, i.e., the energy per delivered bit, as:

$$e_{uv} \;=\; \frac{2\,n_{uv}\,\varphi_{uv}}{c_{uv}^{\mathrm{bw}}\,u_{uv}^{\mathrm{bw}}}\Big[\,P_{\mathrm{port}}\big(u_{uv}^{\mathrm{bw}}\big)\,\big(1+\alpha_r\big)\,\mathrm{PUE}_{uv}^{\mathrm{sites}} + P_{\mathrm{ch}}\big(u_{uv}^{\mathrm{bw}}\big)\,\big(\mathrm{PUE}_{uv}^{\mathrm{sites}} + \alpha_{\mathrm{ch}}\,\mathrm{PUE}_{uv}^{\mathrm{line}}\big)\Big]\,[\mathrm{J/bit}]. \tag{12}$$

Together with (8), (12) constitutes the basis for the service distribution analysis of Section VII.

5) Link Delay:

Besides power, each link contributes delay to the services traversing it, with two components. The *propagation delay* τ_{uv} is deterministic, set by the fiber route at ≈ 4.9 μs/km [38]: with typical route lengths of 20–50 km for metro access–edge links and 1000–3000 km for continental edge–core routes [36], $\tau_{uv} \approx 0.1$–0.25 ms and 5–15 ms one-way, respectively; where edge sites are not present in every city, access–edge routes extend to 100–600 km and τ_{uv} to 0.5–2.5 ms. On top of τ_{uv}, each of the n_{uv} segment terminations adds a load-dependent queueing and transmission delay at its port, kept to sub-millisecond values by the QoS cap on u_{uv}^{bw}.

Even if not explicitly represented in the three-tier network hierarchy considered in this work, we further account for the access-segment delay between the user device and its access site, 5–15 ms round-trip over consumer radio access and 2–5 ms under ultra-reliable low-latency (URLLC) configurations [39], yielding total round-trip link delays ranging from 2–15 ms for access-side serving, to 5–20 ms for edge-side serving, and 25–50 ms when traversing the full path to the core.

Additional components of the end-to-end service latency (response time) — the payload transmission time, the service-dependent processing delay, and the number of network round trips per interaction (e.g., handshakes) — are introduced in Section V to complete the response-time model constrained in the service distribution problem of Section VI.

E. Summary: The Infrastructure Nature

Tables I and II summarize the infrastructure model parameters introduced in this section, with typical ranges anchored in the measurement and modeling literature.

The evaluations of Section VIII establish trends and trade-off conditions across these ranges, as well as insights for representative operating points.

Together, the site and link characterizations instantiate the attributes of the cloud-network graph $\mathcal{G}$. Each site $u \in \mathcal{V}^c$ carries arithmetic and memory capacities (C_u, M_u) and power P_u (7), with specific contributions from different types of accelerator clusters hosting different AI inference models (more on this in the next Section). And each link $(u, v) \in \mathcal{E}^c$ carries capacity C_{uv} and power P_{uv} (11), with no distinction on the nature of carried bits.

Finally, the derived energy efficiency coefficients $e_{u,p}^m$ [J/FLOP] and e_{uv} [J/bit] constitute the infrastructure-side interface to the theoretical analysis of Section VII.

After defining the AI infrastructure side key models, parameters, and variables, we now proceed with the same task for the AI inference service side.

V. AI Inference Model

We particularize the generic service model of Section III to representative model for AI inference services. We first define the architectural structure of an inference service and its representation as a service graph $\mathcal{R}$. We then characterize the computational, memory, and communication requirements of its constituent functions and map them to the corresponding commodity attributes in $\mathcal{R}$. Finally, we introduce three workload factors that capture the intrinsic resource profile of an inference task, independently of its offered load and the underlying hardware platform.

A. Architectural Structure

We characterize every inference function along two orthogonal dimensions:

- the *inference paradigm*, i.e., the computational process through which the output is generated;

[3] Such mean PUE is also applied to the terminations at intermediate points of presence when a link comprises multiple segments.

- the *backbone architecture*, i.e., the neural network(s) that implements evaluation within that process.

1) Inference paradigm: Contemporary foundation models fall into three main paradigms.

Autoregressive inference factorizes the joint output distribution into a product of conditionals. It generates output tokens sequentially, each conditioned on all previous ones [40], [41]. To avoid recomputation, the keys and values of past positions are kept in a key–value (KV) cache whose size grows with the resident context. Execution has two sub-phases:

- *prefill*, in which the T_{in} input tokens are processed in parallel and the cache is populated;
- *decode*, in which the T_{out} output tokens are generated one at a time, each reading and extending the cache.

The cache makes the paradigm stateful.

Diffusion (or flow-based) inference synthesize high-dimensional outputs by evaluating N_{steps} iterative denoising or trajectory steps over a constant-dimensional latent representation [42], [43]. Because each step updates the entire latent tensor concurrently, execution involves no state other than the fixed-size latent, which is updated in place. Activation memory is fully overwritten and reused step-to-step, rendering the paradigm strictly stateless with dynamic memory requirements bounded independently of N_{steps}. Upon trajectory completion, a single terminal pass through a signal decoder projects the refined latent state back to the target physical domain.

Discriminative (or analytics) inference maps one or more input modalities to a bounded structured output, such as labels, bounding boxes, masks, embeddings, or a rendered overlay. It uses a single forward evaluation per input sample, with neither sequential generation nor iterative refinement. The paradigm is stateless, and its activation memory is bounded independently of the output.

2) Backbone architecture: We refer to the *backbone* as the neural network that carries the principal computation loop of the inference paradigm and produces the output in its internal representation. The backbone is evaluated according to the execution pattern of the inference paradigm: during both prefill and decode in autoregressive inference, at each denoising (or flow) step in diffusion inference, and once per input in conventional discriminative inference.

A *Transformer* backbone consists of a stack of attention-based blocks that iteratively transform input representations into contextualized features. Each block typically comprises multi-head self-attention, a position-wise feed-forward network, residual connections, and normalization layers [40]. Depending on the model, the backbone may operate on token sequences, latent representations, or modality-specific feature sequences. Transformers constitute a predominant backbone family across the three inference paradigms considered in this work.

Other backbone, such as *Convolutional neural networks* (CNNs), including U-Net architectures [44], remain prevalent in real-time perception and in latent-diffusion generators [45]. Autoregressive convolutional architectures also exist (e.g., WaveNet [46]) but are less representative of contemporary foundation-model deployments.

3) Input/Output modality: Depending on the task, a model may receive text, images, audio, or video as input, and may produce outputs in the same or a different modality. However, the input and output modalities alone do not determine the architectural class of an inference task. In contemporary AI systems, certain modality transformations are commonly associated with specific inference paradigms and backbone architectures:

- Text-to-text (T2T): predominantly autoregressive Transformer models;
- Text-to-Image (T2I) and Text-to-video (T2V): diffusion or flow-based models, increasingly employing Transformer backbones;
- Speech-to-text (S2T): predominantly encoder–decoder or encoder-based speech recognition models, with autoregressive decoding used in some architectures;
- Audio-to-Audio (A2A): autoregressive or diffusion-based models operating over neural-codec tokens;
- Video Analytics (VID (V2T)) and Augmented Reality (AR (V2X)): discriminative CNN- and Transformer-based pipelines.

These associations describe common architectural patterns rather than strict correspondences: the same input/output modality can be supported by different inference paradigms and backbone architectures.

4) AI Model Classification: The Cartesian product of paradigm and backbone gives the classification of Fig. 3. The resource behavior considered in this work—namely, execution pattern, dynamic-memory behavior, and state persistence—is primarily determined by the inference paradigm, whereas the backbone architecture determines the computational and memory cost of each inference evaluation. Accordingly, the paradigm serves as the primary index for the resource models developed below, while the backbone parameters determine the per-evaluation workload.

B. Computation Resource Requirements

We characterize the computational requirements of an inference task by two dimensions: the *processing demand* and the *memory demand*.

We distinguish the parameters exercised during one invocation of the inference neural network, Φ_{dn}, from the total resident parameters, Φ_{tot}. The former determines the arithmetic workload; the latter, the set of weights that must reside in memory, determines the model's static footprint.

1) Processing Requirements: The processing cost of an inference task is determined by the number of floating-point operations required to execute its neural network inference model[4].

Proposition 1 (Generic Computational Model of a Transformer Evaluation)**.** *Consider one evaluation of a Transformer with L layers, width d, and Φ_{dn} activated parameters,*

[4]All FLOP expressions adopt the standard accounting of the scaling-law literature [47], [48]: one multiply–accumulate (MAC) counts as two FLOPs; only matrix multiplications are counted, since element-wise operations, normalizations, and softmax contribute lower-order terms [47]; and FLOPs measure the arithmetic work of the algorithm, independently of hardware utilization.

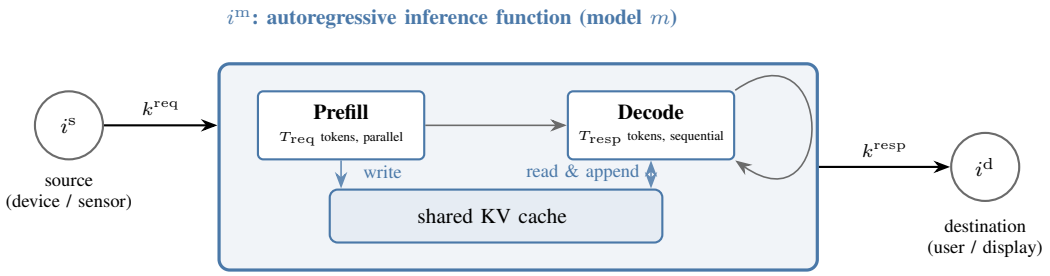


Fig. 2: AI Inference Service Graph representation.

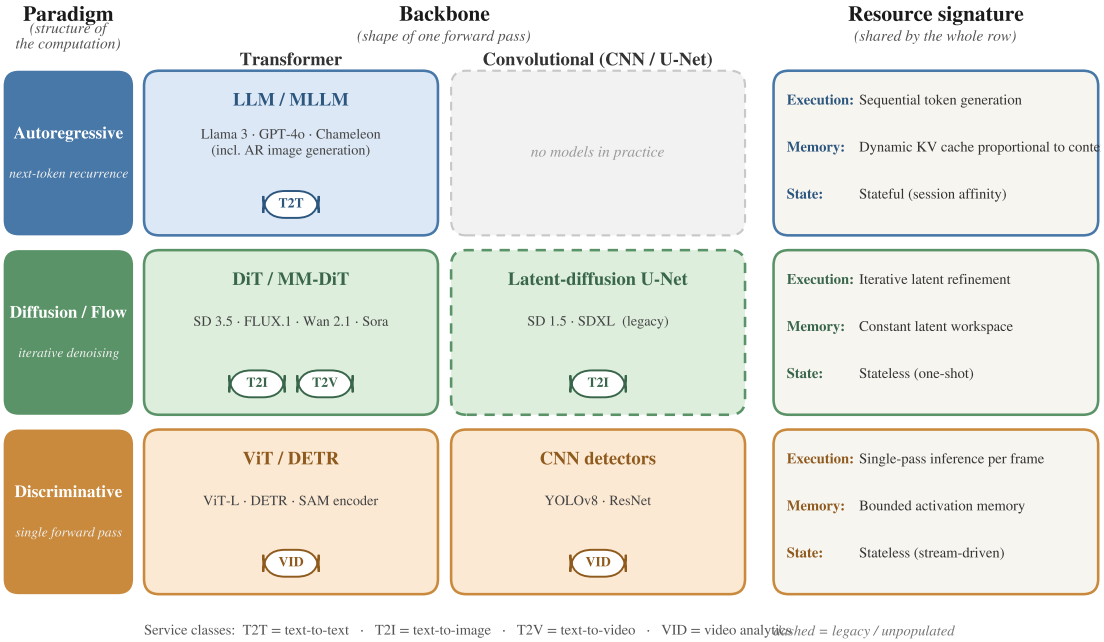


Fig. 3: Classification of AI inference functions by inference paradigm (rows) and backbone architecture (columns). Each row shares a resource signature (execution, dynamic memory, state). Service classes: T2T (text-to-text), T2I (text-to-image), T2V (text-to-video), S2T (speech-to-text), A2A (speech-to-speech), VID (video analytics), AR (augmented reality). Dashed cells are legacy or rarely deployed.

evaluating an input sequence comprising n tokens with an attention pattern comprising A query–key pairs. Under the stated counting convention, the evaluation requires

$$\mathrm{FLOPs} = \underbrace{2\,\Phi_{\mathrm{dn}}\,n}_{\textit{parameter term}} + \underbrace{4\,L\,d\,A}_{\textit{attention term}} + O(n), \qquad (13)$$

where the parameter term accounts for one multiply and one add per parameter per token [47], and the attention term counts the two attention matrix products—query–key scores and attention-weighted values—each costing d MACs per pair.[5]

Transformer evaluation is determined by the number of processed tokens and the attention pattern connecting them.

[5]The linear projections producing Q, K, V and the attention output are parameterized and hence already included in the parameter term; only the two parameter-free matrix products constitute the additional, sequence-dependent cost.

TABLE III: Backbone quantities of (13) per paradigm

Paradigm	Evaluations	Tokens n	Query-key pairs per layer A
Autoregr. (per turn)	1 (aggregate)	S	$\frac{S(S+1)}{2}$
Diffusion	$N_{\mathrm{steps}}\,g$	$T_{\mathrm{lat}} + T_{\mathrm{txt}}$	$n^2 + n\,T_{\times}$
Discriminative	1	T_{in}	T_{in}^2

The paradigms differ in these two quantities, as summarized in Table III and derived next.

a) Autoregressive paradigm: Let T_{req} and T_{resp} denote the input and generated token counts of a single turn, and $S \triangleq T_{\mathrm{req}} + T_{\mathrm{resp}}$. Owing to KV caching, each token is evaluated exactly once: prefill processes the T_{req} input tokens in parallel, and each decode step computes only the newly generated token, reusing the cached keys and values of its prefix [49]. Hence $n = S$ tokens are processed under a causal

pattern with $A = S(S{+}1)/2$ pairs, and Proposition 1 yields

$$\begin{aligned}\mathrm{FLOPs}_{\mathrm{AR}} &= \underbrace{2\,\Phi_{\mathrm{dn}}\,T_{\mathrm{req}}}_{\text{prefill}} + \underbrace{2\,\Phi_{\mathrm{dn}}\,T_{\mathrm{resp}}}_{\text{decode}} + \underbrace{2\,L\,d\,S\,(S{+}1)}_{\text{attention}} \\ &\approx\; 2\,\Phi_{\mathrm{dn}}\,S \;+\; 2\,L\,d\,S^2.\end{aligned} \tag{14}$$

We also note that, in addition to the backbone, an inference function may comprises other several neural networks. These components account for input encoders, auxiliary or conditioning networks, and output decoders or task-specific heads. Depending on the specific model architecture, any of these subsets may be empty. In this work, we primarily focus on backbones and, where needed, we include detailed computational and resource characterization of each peripheral network.

b) Diffusion paradigm: Unlike autoregressive models, diffusion and flow-based models repeatedly evaluate the same denoising network over a fixed latent representation. Diffusion and flow-matching models synthesize the output through N_{steps} iterative denoising evaluations of the backbone [50]–[52], each a *full bidirectional* pass over a fixed grid of T_{vis} latent tokens: $n = T_{\mathrm{vis}}$ and $A = T_{\mathrm{vis}}^2$ per evaluation. Classifier-free guidance evaluates the denoiser on a conditional and an unconditional branch per step [53]. This doubles the number of denoiser evaluations and is captured by the multiplicative guidance factor g ($g = 2$ under standard guidance; $g = 1$ for guidance-distilled models such as FLUX.1-dev [54]). Adding one text-encoder pass over the T_{txt} prompt tokens and one VAE decode of cost C_{vae},

$$\begin{aligned}&\mathrm{FLOPs}_{\mathrm{diff}} = \\ &N_{\mathrm{steps}}\,g\left(2\,\Phi_{\mathrm{dn}}\,T_{\mathrm{vis}} + 4\,L\,d\,T_{\mathrm{vis}}^2\right) + \underbrace{2\,\Phi_{\mathrm{enc}}\,T_{\mathrm{txt}} + C_{\mathrm{vae}}}_{\text{encoder and VAE, once per request}}\end{aligned} \tag{15}$$

The structural contrasts with (14) are the multiplicative factor $N_{\mathrm{steps}}\,g$, the absence of any generated-token count, and the bidirectional attention pattern, whose pair count, T_{vis}^2, is approximately twice the $T_{\mathrm{vis}}(T_{\mathrm{vis}}{+}1)/2$ pairs of a causal stream of equal length.[6]

The latent token count follows from the compression chain. A VAE with spatial factor f compresses $H \times W$ pixels to a $(H/f) \times (W/f)$ latent grid [56], which the Transformer patchifies with patch size p [57]:

$$T_{\mathrm{vis}} = \frac{H\,W}{(f\,p)^2}. \tag{16}$$

Typical operating points are $f = 8$ and $p = 2$. For video, a causal 3D VAE additionally compresses the temporal axis by a factor c_t [58], so that a clip of N_f frames occupies $\lceil (N_f{-}1)/c_t \rceil + 1$ latent frames and

$$T_{\mathrm{vis}} \;\approx\; \frac{N_f}{c_t} \cdot \frac{H\,W}{(f\,p)^2}. \tag{17}$$

[6]In MM-DiT architectures the prompt tokens join the denoiser's attention sequence [55]; the exact token count is $T_{\mathrm{vis}} + T_{\mathrm{txt}}$, a sub-percent correction for $T_{\mathrm{txt}} \sim 10^2 \ll T_{\mathrm{vis}}$ that we omit.

c) Discriminative paradigm: A discriminative model executes a single forward pass per input frame. For a ViT backbone, the cost is Proposition 1 with $n = T_{\mathrm{img}}$ and $A = T_{\mathrm{img}}^2$, evaluated once. For a convolutional backbone, the per-frame cost is the forward complexity at the operating resolution [59],

$$\mathrm{FLOPs}_{\mathrm{disc}} = \Psi(H, W) = 2\sum_{l=1}^{L} k_l^2\; c_l^{\mathrm{in}}\; c_l^{\mathrm{out}}\; H_l\, W_l, \tag{18}$$

where k_l, c_l^{in}, c_l^{out}, and $H_l \times W_l$ denote the kernel size, channel counts, and output feature-map dimensions of layer l. In practice Ψ is reported directly on the model card; because the cost scales with the spatial extent of the feature maps, it rescales to any camera resolution as $\Psi(H, W) \approx \Psi(H_0, W_0) \cdot HW/(H_0 W_0)$, an approximation exact for fully convolutional architectures and accurate for detection backbones without resolution-fixed heads.

d) CNFlow abstraction: Within the CNFlow framework, the processing requirement of a service is carried by the rate $R^{k,\mathrm{flop}}$ of the corresponding commodity k, obtained by scaling the per-unit cost by the drive rate of the class:

$$\boxed{R^{k,\mathrm{flop}} = \mathrm{FLOPs}_{\bullet} \cdot \lambda, \qquad \lambda = \begin{cases} \lambda_{\mathrm{req}} & \text{request-driven} \\ f \cdot N_{\mathrm{sensors}} & \text{stream-driven} \end{cases}} \tag{19}$$

with $\mathrm{FLOPs}_{\bullet}$ given by (14), (15), or (18) according to the paradigm and denoting the per-unit processing cost (FLOP per request for request-driven classes, FLOP per frame for stream-driven), λ_{req} the aggregate request arrival rate at the ingress node to which the source function of the service graph is mapped, f the frame rate, and N_{sensors} the number of admitted streams. Note that, for long-term optimization algorithms, the FLOPs per request times the request arrival rate equals the FLOPs per conversation times the conversation arrival rate. This is because a conversation arriving at rate λ_{conv} carries on average N_t turns, so requests are injected at $\lambda_{\mathrm{req}} = N_t \cdot \lambda_{\mathrm{conv}}$, while its total arithmetic is the sum over those turns, $\mathrm{FLOP}_{\mathrm{conv}} = N_t\,\mathrm{FLOP}_{\mathrm{req}}$. The factor N_t therefore multiplies the rate and divides the per-unit cost, and cancels: the processing rate $R^{k,\mathrm{flop}}$ is invariant to whether it is computed at request or conversation granularity. This granularity-freedom is a property of flows (work per unit time); the memory requirement $R^{k,\mathrm{vram}}$ does not share it, since it is pinned to the conversation lifetime $\bar{\tau}_{\mathrm{conv}}$ (Sec. V-B2c).

Furthermore, (19) is the only place where load enters: the per-unit costs (14)–(18) are load-independent workload invariants, a property exploited by the service distribution analysis of Sec. VII.

2) VRAM Requirements: Unlike processing requirements, which are entirely determined by the execution of the inference model, GPU memory (VRAM) comprises two fundamentally different components. A static component stores the model parameters and is allocated once per deployed replica, whereas a dynamic component stores execution state associated with individual units of work. These two components exhibit different scaling laws and therefore enter the CNFlow resource model through distinct allocation mechanisms.

a) Static footprint: For a model with Φ_{tot} resident parameters stored at numerical precision Q bits per parameter, the static GPU memory required by a deployed model replica equals

$$M_{\text{model}} = \Phi_{\text{tot}}\, Q \quad \text{[bit]}, \tag{20}$$

with $Q = 16$ for the FP16/BF16 deployments considered here and $Q \in \{8, 4\}$ under 8- and 4-bit weight quantization [60]. M_{model} is independent of the number of active requests and of their context lengths: it is a *per-replica* quantity, incurred once when the model is instantiated at a computation node and amortized across all work served there.

b) Dynamic footprint: The dynamic footprint depends on the execution state maintained by the inference paradigm. Accordingly, autoregressive, diffusion, and discriminative inference exhibit distinct memory behaviors despite sharing the same static model footprint.

Autoregressive paradigm. During autoregressive decoding, the key and value projections of previously processed tokens are retained in GPU memory to avoid recomputation during subsequent decoding steps. This persistent key–value (KV) cache constitutes the dominant dynamic memory allocation of autoregressive inference [49]. Each resident token contributes, per layer, one key and one value vector of dimension $n_{kv}\, d_{\text{head}}$, where n_{kv} is the number of distinct key–value heads; in standard multi-head attention n_{kv} equals the number of query heads, while multi-query and grouped-query attention share KV heads across query heads precisely to shrink this footprint [61], [62] (e.g., $n_{kv} = 8$ for Llama-3, Table VIII). The cache cost per resident token is therefore

$$m_{\text{tok}} = 2\, L\, n_{kv}\, d_{\text{head}}\, Q, \tag{21}$$

where the factor of two accounts for storing both keys and values. Therefore, a single-turn interaction of $S = T_{\text{req}} + T_{\text{resp}}$ tokens holds

$$M_{\text{dyn}}^{ar,req} = m_{\text{tok}}\, S = 2\,(T_{\text{req}} + T_{\text{resp}})\, L\, n_{kv}\, d_{\text{head}}\, Q. \tag{22}$$

As established in Sec. V-A1, autoregressive inference is stateful: the cache persists across the turns of a conversation, holding the union of resident tokens, and grows until truncated at the model's context window $C_{\max}$. For long-term optimization algorithms, infrastructure provisioning depends on the average amount of resident context maintained across a conversation rather than on the instantaneous cache size. Consider a conversation of N_{t} turns of mean length S tokens; the cache holds jS tokens after turn j, so the session-averaged resident context is

$$\begin{aligned} \bar{T}_{\text{ctx}} &= \frac{1}{N_{\text{t}}} \sum_{j=1}^{N_{\text{t}}} \min(jS,\, C_{\max}) \\ &\approx \frac{N_{\text{t}} + 1}{2}\, S \quad \text{when } N_{\text{t}} S \le C_{\max}, \end{aligned} \tag{23}$$

and saturates toward $C_{\max}$ for long conversations. The dynamic footprint of the autoregressive class is then

$$M_{\text{dyn}}^{\text{ar,conv}} = m_{\text{tok}}\, \bar{T}_{\text{ctx}}, \tag{24}$$

which recovers the single-turn cost (22) for stateless one-shot use ($N_{\text{t}} = 1$) and scales linearly with conversation depth until the context window binds.

Diffusion paradigm. Diffusion inference is stateless. Since denoising proceeds over a fixed latent representation, no information from previous iterations must be retained beyond the current activation workspace. Denoising is non-autoregressive, hence the activation workspace of one denoiser evaluation is released and reused across all N_{steps} iterations. The dynamic footprint therefore consists of the persistent latent state, of size $T_{\text{vis}}\, d\, Q$, plus the simultaneously-live activation buffers of one evaluation—a small multiple ν of the token representation, doubled to g parallel branches under classifier-free guidance [53]:

$$M_{\text{dyn}}^{\text{diff}} \approx (1 + \nu g)\, T_{\text{vis}}\, d\, Q \quad \text{[bit per request]}, \tag{25}$$

where ν is an implementation-dependent constant accounting for the activation tensors of size $O(T_{\text{vis}} d)$ held live per block (residual stream, attention projections, MLP hidden states, modulated by kernel fusion). Typical values are $\nu \in [2, 4]$ for modern inference engines.

Equation (25) reveals the defining property of diffusion inference: the dynamic memory footprint is independent of the number of denoising iterations. Additional iterations increase computation but not memory consumption because the same workspace is reused throughout the inference process.

Discriminative paradigm. A discriminative model maintains neither a cache (it is non-autoregressive) nor a persistent denoising workspace (there is no step loop); the only per-work allocation is the activation memory of a single forward pass,

$$M_{\text{dyn}}^{\text{disc}} \;\approx\; \nu a\, A_{\text{peak}}\, Q, \tag{26}$$

where A_{peak} is the peak activation tensor of the forward pass at the operating resolution and νa accounts for buffers simultaneously live [59]. $M_{\text{dyn}}^{\text{disc}}$ is bounded and independent of any output length.

c) CNFlow abstraction: While M_{model} is incurred once per replica, the dynamic footprint is incurred per unit of work *for as long as that work holds its allocation*. Let $\bar{\tau}$ denote this memory holding time. Because dynamic memory is a *stock*—bytes occupied over an interval—rather than a flow, its aggregate demand is obtained from the offered rate and the holding time through Little's law [63]: the number of concurrently held memory allocations is $N_{\text{conc}} = \lambda \bar{\tau}$. The pair $(\lambda, \bar{\tau})$ must be interpreted at the granularity of the work unit that *holds* the state, which the inference paradigm determines:

- **Stateful request-driven (autoregressive).** The KV cache is retained across the turns of a conversation, so the state-holding unit is the conversation: $\lambda = \lambda_{\text{conv}}$ is the session arrival rate [conversations/s] and $\bar{\tau} = \bar{\tau}_{\text{conv}}$[s per conversations] is the session holding time—the interval over which the cache is maintained in VRAM for a given conversation, spanning both its active turns and the inter-turn idle gaps.
- **Stateless request-driven (diffusion, discriminative one-shot).** No state persists beyond a single request, so the

TABLE IV: Encoded size B_i of each item type.

Item	B_i [bit]	Services
Text (T tokens)	$b_{\text{tok}}\,T$	T2T, T2I/T2V input, S2T output
Audio clip (bitrate β, duration D)	$\beta\,D$	S2T input, A2A
Generated image or video	encoded file size	T2I/T2V output
Stream frame (bitrate β)	β/f_{a}	VID/AR input
Analytics output per frame	predictions or $\beta_{\text{out}}/f_{\text{a}}$	VID/AR output

work unit is the request: $\lambda = \lambda_{\text{req}}$ and $\bar{\tau} = \tau_{\text{req}}$ is time to process one request (work unit).

- **Stream-driven (discriminative).** Stream units (e.g., frames) arrive continuously at rate $\lambda = f N_{\text{sensors}}$, where f is the frame rate and N_{sensors} the number of admitted streams, and $\bar{\tau} = \tau_{\text{req}}$ is the per-frame service time. The holding time equals the service time provided $\tau_{\text{req}} < 1/f$, i.e., the service time stays below the inter-frame interval (≈ 33 ms at 30 fps), so that no queue accumulates and each frame's allocation is released before the next arrives.

In all cases the CNFlow dynamic-memory demand of commodity k follows as:

$$\boxed{\begin{aligned} R^{k,\text{vram}} &= M_{\text{dyn}}\,N_{\text{conc}} = M_{\text{dyn}}\,\lambda\,\bar{\tau}, \\ M_{\text{dyn}} &= \begin{cases} M_{\text{dyn}}^{ar,conv} & \text{autoregressive} \\ M_{\text{dyn}}^{\text{diff}} & \text{diffusion} \\ M_{\text{dyn}}^{\text{disc}} & \text{discriminative} \end{cases} \end{aligned}} \tag{27}$$

C. Response time

Time to first token (What are all the components that influence the time to first token (time to request arrive to the service + time to process + time to send back to the user).

We assume the time to process as QoS. In order to controll the middle one we use the target utilization of the gpu that we introduce in sec before.

D. Communication Resource Requirements

Unlike processing and memory requirements, which depend on the internal execution of the inference model, communication requirements are determined by the product of its payload size B [bit] and the arrival rate λ_k.

$$\boxed{R_{\text{req}}^{\text{bw}} = B_{\text{in}}\,\lambda, \qquad R_{\text{resp}}^{\text{bw}} = B_{\text{out}}\,\lambda \quad [\text{bit/s}].} \tag{28}$$

with B_i depending on the type of media transmitted to the network, given in Table IV.

E. Relative Resource Factors

The analytical models developed in the previous sections quantify the absolute processing, memory, and communication requirements of an inference workload. While these quantities determine the amount of resources consumed, placement decisions depend primarily on the relative balance among resource dimensions. This motivates the introduction of three dimensionless workload descriptors that summarize the intrinsic resource profile of an inference task independently of its offered load and of the underlying hardware platform.

Definition 1 (Computation-to-Communication Ratio)**.** *The Computation-to-Communication Ratio measures the amount of computation performed per transmitted bit.*

$$\boxed{I \;\triangleq\; \frac{R^{\text{flop}}}{R_{\text{req}}^{\text{bw}} + R_{\text{resp}}^{\text{bw}}}\;\; [\text{FLOPS/bps}],} \tag{29}$$

High values identify computation-intensive workloads, whereas low values identify communication-intensive workloads.

Definition 2 (Computation-to-Memory Ratio)**.** *The computation-to-memory ratio measures the amount of computation performed per unit of dynamic memory held, i.e., the arithmetic throughput a workload demands relative to the accelerator memory its execution state occupies:*

$$\boxed{\sigma \;\triangleq\; \frac{R^{\text{flop}}}{R^{\text{vram}}} \qquad [\text{FLOP/s}\cdot\text{bit}^{-1}] \equiv [\text{FLOPS/bit}].} \tag{30}$$

Expanding the two rates through their CNFlow definitions (Eqs. (19) *and* (27)*), the offered load λ cancels and σ reduces to a per-work-unit quantity, the arithmetic work of one unit divided by the memory–time it holds on the accelerator.*

$$\sigma = \frac{\text{FLOPs}\,\lambda}{M_{\text{dyn}}\,N_{\text{conc}}} = \frac{\text{FLOPs}\,\lambda}{M_{\text{dyn}}\,\lambda\,\bar{\tau}} = \frac{\text{FLOPs}}{M_{\text{dyn}}\,\bar{\tau}}, \tag{31}$$

Equation (31) *shows that σ decreases with the holding time $\bar{\tau}$: the longer a unit of work must keep its state resident, the more memory it ties up per FLOP, and the smaller σ becomes.*

Definition 3 (Response-to-request ratio)**.** *The response-to-request ratio is the ratio of downlink to uplink communication demand,*

$$\boxed{\rho \;\triangleq\; \frac{R_{\text{resp}}^{\text{bw}}}{R_{\text{req}}^{\text{bw}}} \;=\; \frac{B_{\text{resp}}}{B_{\text{req}}}.} \tag{32}$$

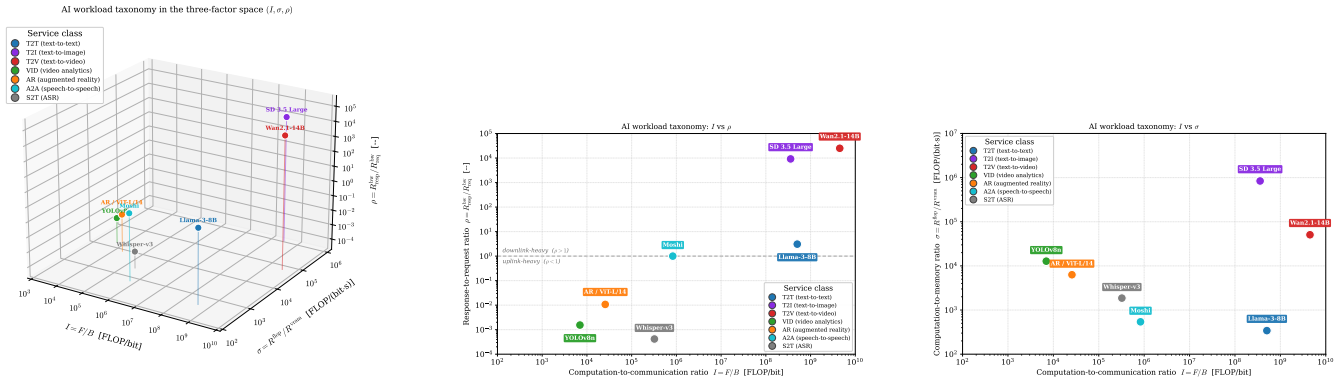
ρ captures the directional asymmetry of a workload's traffic.

The three factors (I, σ, ρ) form a complementary, hardware-independent coordinate system for AI inference workloads. Each axis captures a distinct aspect of the workload resource profile:

- I characterizes the balance between computation and communication requirements;
- σ characterizes the balance between computation and memory requirements;
- ρ characterizes the directional asymmetry between outputs and inputs rates.

Since each captures a different aspect of the resource profile, every inference workload can be represented as a point (I, σ, ρ) in a three-dimensional resource space. This representation is independent of implementation details and serves as the basis for the workload taxonomy and placement analysis developed in the remainder of the paper, as illustrated in Figure 4.

We summarize the per-request processing, memory, and communication requirements in Table V, and the aggregate per-deployment rates in Table VI. Further detail on the representative foundation models and their inference workloads is provided in Appendix A.



(a) 3D Factor Space. (b) I vs Rho (c) I vs Sigma

Fig. 4

TABLE V: Per-request processing, memory and communication requirements

Model	λ	$\bar{\tau}$ [s]	FLOPs [GFLOP]	M_{model} [MB]	M_{dyn} [MB]	N_{conc}	B_{in} [Mbit]	B_{out} [Mbit]
Llama-3-8B	765	60.0	9148	1.61×10^4	55.7	4.59×10^4	4.45×10^{-3}	0.0137
SD 3.5 Large	1.75	5.00	4.01×10^6	2.74×10^4	120	8.75	1.22×10^{-3}	11.2
Wan2.1-14B	0.0390	220	1.80×10^8	3.93×10^4	2013	8.58	1.60×10^{-3}	40.0
YOLOv8n	8.10×10^5	5.00×10^{-3}	8.70	6.40	17.0	4050	1.24	1.92×10^{-3}
AR / ViT-L/14	8.40×10^4	8.00×10^{-3}	80.7	608	200	672	3.11	0.0333
Whisper-v3	2820	1.00	2480	3084	166	2820	7.68	3.20×10^{-3}
Moshi	3290	30.0	2127	1.54×10^4	16.4	9.87×10^4	1.28	1.28

TABLE VI: Aggregate per-deployment demand rates.

Model	R^{flop} [GF/s]	R^{vram} [MB]	$R^{\text{bw}}_{\text{in}}$ [Mbps]	$R^{\text{bw}}_{\text{out}}$ [Mbps]
Llama-3-8B	7.00×10^6	2.56×10^6	3.40	10.5
SD 3.5 Large	7.02×10^6	1046	2.13×10^{-3}	19.6
Wan2.1-14B	7.01×10^6	1.73×10^4	6.24×10^{-5}	1.56
YOLOv8n	7.05×10^6	6.88×10^4	1.01×10^6	1555
AR / ViT-L/14	6.78×10^6	1.34×10^5	2.61×10^5	2800
Whisper-v3	6.99×10^6	4.69×10^5	2.17×10^4	9.02
Moshi	7.00×10^6	1.62×10^6	4211	4211

VI. Problem Formulation

Given a full description of a AI Network Infrastructure cloud-network graph $\mathcal{G}$ and a full description of a Multimodal LLM service graph $\mathcal{R}$, we solve the AI-PROBLEM as finding (i) the number of replicas and location of each service function $i \in \mathcal{I}$, (ii) the routes of each request and response commodity, and (iii) the allocation of compute, storage, and communication resources.

The CNFlow is subject to the following constraints:

VII. Theoretical Analysis

The analytical models developed in the previous sections quantify the processing, memory, and communication requirements of an inference workload in terms of the intrinsic factors introduced in Sec. V-E. Building on these models, we investigate how these factors influence the energy-optimal placement of an AI service. In particular, we derive a closed-form characterization of the optimal placement for a given accelerator–model pair.

We consider a model m and a GPU a available at every tier of the access–edge–core hierarchy described in Sec. IV, with source and destination functions placed at the access tier. In this section, we first derive the effective capacity of the accelerator. We then derive the energy efficiency of computation and transport at each tier. Finally, we obtain closed-form thresholds on the computation-to-communication ratio I that determine the optimal tier.

A. Effective Accelerator Capacity

An accelerator a hosting model m provides two resources at once: arithmetic throughput (FLOPS) and memory (VRAM). It can therefore serve work only until the first of the two resources is exhausted. Two cases arise:

- The arithmetic limit is set by the target utilization: once the processing rate served by the accelerator reaches $c^{\text{flop}}u$, a new accelerator is needed.
- The memory limit is set by the available VRAM of the GPU: once the dynamic memory requirement reaches $c^{\text{vram}} - M_{\text{model}}$, a new accelerator is needed.

The following lemma [...]

Lemma 1 (Effective capacity)**.** *Let a be an accelerator with compute capacity c^{flop} [FLOPS], target utilization u,*

$$\textbf{min} \sum_{u\in\mathcal{V}^c} P_u \;+\; \sum_{(u,v)\in\mathcal{E}^c} P_{uv} \tag{33}$$

$$P_u = \mathrm{PUE}_u\,\varphi_u \sum_{p\in\mathcal{V}^p(u)} \big(1+\alpha_p\big) \sum_{m\in\mathcal{M}} P_p^m \qquad \forall u\in\mathcal{V}^c \tag{34}$$

$$P_p^m = y_p^m\, P_{\mathrm{idle},a(p)} + y_p^m u_{a(p),m}^{\mathrm{flop}} \big(P_{\mathrm{TDP},a(p)} - P_{\mathrm{idle},a(p)}\big) \qquad \forall p,m \tag{35}$$

$$P_p^m = y_p^m\, P_{\mathrm{idle},a(p)} + \frac{\mu_p^{m,\mathrm{flop}}}{c_{a(p)}^{\mathrm{flop}}} \big(P_{\mathrm{TDP},a(p)} - P_{\mathrm{idle},a(p)}\big) \qquad \forall p,m \tag{36}$$

$$P_{uv} = y_{uv}\, n_{uv}\, \varphi_{uv} \Big[2\, P_{uv}^{\mathrm{port}} \big(1+\alpha_r\big)\mathrm{PUE}_{uv}^{\mathrm{sites}} + 2\, P_{uv}^{\mathrm{ch}} \big(\mathrm{PUE}_{uv}^{\mathrm{sites}} + \alpha_{\mathrm{ch}}\, \mathrm{PUE}_{uv}^{\mathrm{line}}\big)\Big] \qquad \forall (u,v)\in\mathcal{E}^c \tag{37}$$

$$\sum_{v\in\mathcal{N}^-(u)} f_{vu}^k = \sum_{v\in\mathcal{N}^+(u)} f_{uv}^k \qquad \forall u\in\mathcal{V},\; k\in\mathcal{K} \tag{38}$$

$$f_{uv}^k = \begin{cases} f_{vu}^\ell & \forall k\in\mathcal{K}^p \\ 0 & \text{o.w.} \end{cases} \qquad \forall \ell\in\mathcal{X}(k),\, u\in\mathcal{V}^{p,\mathcal{K}}(k),\, v\in\mathcal{N}^+(u) \tag{39}$$

$$f_{uv}^k = \begin{cases} 1 & \forall k\in\mathcal{K}^s \\ 0 & \text{o.w.} \end{cases} \qquad \forall u = s^{\mathcal{K}}(k),\, v\in\mathcal{N}^+(u) \tag{40}$$

$$f_{uv}^k = \begin{cases} 1 & \forall k\in\mathcal{K}^d \\ 0 & \text{o.w.} \end{cases} \qquad \forall v = d^{\mathcal{K}}(k),\, u\in\mathcal{N}^-(v) \tag{41}$$

$$\sum_{k\in K} f_{uv}^k R^{\mathrm{k,bw}} = \mu_{uv}^{bw} \qquad \forall (u,v)\in\mathcal{E}^c \tag{42}$$

$$\sum_{k\in\mathcal{K}^+(m)} f_{pu}^k R^{\mathrm{k,flop}} \frac{1}{MFU_{p,m}} = \mu_p^{m,\mathrm{flop}} \qquad \forall (p,u)\in\mathcal{E}^{p+},\; m\in M \tag{43}$$

$$\sum_{k\in\mathcal{K}^-(m)} f_{up}^k R^{\mathrm{k,vram}} = \mu_p^{m,\mathrm{vram}} \qquad \forall (u,p)\in\mathcal{E}^{p-},\; m\in M \tag{44}$$

$$\mu_{uv}^{bw} \le y_{uv}\, c_{uv}^{bw} \qquad \forall (u,v)\in\mathcal{E}^c \tag{45}$$

$$\mu_p^{m,\mathrm{flop}} \le y_p^m (c_{a(p)}^{\mathrm{flop}} \times u_{\mathrm{target}}^m) \qquad \forall p\in\mathcal{V}^p,\; m\in M \tag{46}$$

$$\mu_p^{m,\mathrm{vram}} \le y_p^m (c_{a(p)}^{\mathrm{vram}} - M_{\mathrm{model}}^m) \qquad \forall p\in\mathcal{V}^p,\; m\in M \tag{47}$$

$$\sum_m y_p^m \le c_p \qquad \forall p\in\mathcal{V}^p \tag{48}$$

$$l_{uv}^k = \begin{cases} t^k + \tau_{uv} & \text{if } (u,v)\in\mathcal{E}^c \\ \delta^k & \text{if } (u,v)\in\mathcal{E}^{p+} \\ 0 & \text{otherwise} \end{cases} \qquad \forall (u,v)\in\mathcal{E}, \forall k\in\mathcal{K} \tag{49}$$

$$l^k = \sum_{(u,v)\in\mathcal{E}} l_{uv}^k f_{uv}^k \qquad \forall k\in\mathcal{K} \tag{50}$$

$$l_T^k = l^k \qquad \forall k\in\mathcal{K}^s \tag{51}$$

$$l_T^k \ge l^k + l_T^\ell \qquad \forall k\in\mathcal{K}\backslash\mathcal{K}^s, \ell\in\mathcal{X}(k) \tag{52}$$

$$l_T^k \le L_T^k \qquad \forall k\in\mathcal{K}^d \tag{53}$$

and memory capacity c^{vram} [bit], hosting a model m with static footprint $M_{\mathrm{model}} < c^{\mathrm{vram}}$ and computation-to-memory ratio $\sigma(m)$. Its maximum effective GPU processing throughput [FLOPS] is:

$$\hat{c}(a,m) = \min\Big\{\, c^{\mathrm{flop}} u,\; \big(c^{\mathrm{vram}} - M_{\mathrm{model}}\big)\,\sigma(m) \,\Big\} \quad \textit{[FLOPS]}. \tag{54}$$

Proof. Let x be the service processing rate, in FLOPS, served by accelerator a. Since $1/\sigma(m)$ is the dynamic memory held per FLOPS, we can define the memory that the accelerator holds while serving x FLOPS as $x/\sigma(m)$. Serving x is therefore feasible only if

$$x \le c^{\mathrm{flop}} u \qquad \text{and} \qquad x/\sigma(m) \le c^{\mathrm{vram}} - M_{\mathrm{model}}.$$

As x grows, the accelerator keeps serving until the first of

the two limits is reached, and $\hat{c}$ is the value at which this happens:

Case 1: at $x = c^{\text{flop}}u$ the memory is not yet full, i.e., $c^{\text{flop}}u/\sigma(m) \le c^{\text{vram}} - M_{\text{model}}$. The utilization limit binds first, and $\hat{c}(a,m) = c^{\text{flop}}u$.

Case 2: otherwise, the memory fills first, at $x = (c^{\text{vram}} - M_{\text{model}})\,\sigma(m) < c^{\text{flop}}u$.

In both cases $\hat{c}$ is the smaller of the two limits. □

Definition 4 (Compute-bound and memory-bound)**.** *We define the binding threshold*

$$\sigma^*(a,m) \triangleq \frac{c^{\text{flop}}\,u}{c^{\text{vram}} - M_{\text{model}}}, \tag{55}$$

namely the value of σ at which the two arguments of (54) *coincide. A service with model m is* compute-bound *on GPU a when $\sigma(m) \ge \sigma^*(a,m)$, and then $\hat{c} = c^{\text{flop}}u$. It is* memory-bound *when $\sigma(m) < \sigma^*(a,m)$: its resident state fills the available VRAM before the target utilization is reached, and then $\hat{c} = (c^{\text{vram}} - M_{\text{model}})\sigma < c^{\text{flop}}u$.*

B. Energy Efficiency per Tier

a) Computation: The number of allocated GPUs increases when the active ones reach their effective capacity $\hat{c}(a,m)$. Therefore, each active accelerator operates at utilization $u_{\text{op}} = \hat{c}(a,m)/c^{\text{flop}}$. This equals the target u for compute-bound services and is lower for memory-bound ones.[7] Evaluating (8) at u_{op}, the energy per FLOP executed at tier $t \in \{A, E, C\}$ is

$$\varepsilon_t(a,m) = \text{PUE}_t\,\kappa(a,m), \tag{56}$$

$$\kappa(a,m) \triangleq (1+\alpha)\varphi\Big[\frac{P_{\text{idle}}}{\hat{c}(a,m)} + \frac{P_{\text{TDP}} - P_{\text{idle}}}{c^{\text{flop}}}\Big] \quad \text{[J/FLOP]}. \tag{57}$$

The factor $\kappa(a,m)$ is the energy per FLOP of the accelerator and its host before facility overhead.

b) Transport: Let w_{EA} and w_{CE} [J/bit] denote the energy per delivered bit of (12) on the access–edge and edge–core links. A service placed at tier t transports its request and response over the links between the access tier and t, at an energy per bit of

$$W_A = 0, \quad W_E = w_{EA}, \quad W_C = w_{EA} + w_{CE} \ \text{[J/bit]}. \tag{58}$$

c) Power per tier: A service with processing requirement R^{flop} [FLOPS] and communication requirement $R^{\text{bw}} = R^{\text{bw}}_{\text{req}} + R^{\text{bw}}_{\text{resp}}$ [bit/s], placed at tier t, draws the power

$$\begin{aligned} P_t &= R^{\text{flop}}\varepsilon_t(a,m) + R^{\text{bw}}W_t \\ &= R^{\text{bw}}\big[\, I\,\varepsilon_t(a,m) + W_t\,\big] \quad \text{[W]}, \end{aligned} \tag{59}$$

with $I = R^{\text{flop}}/R^{\text{bw}}$ [FLOP/bit] as in Definition 1. From (59), the optimal tier depends on the workload only through I and, via $\hat{c}(a,m)$, through $\sigma(m)$ and M_{model}. It does not depend on the offered load λ.

[7] All active accelerators are fully loaded except possibly the last one. Its residual idle draw is at most P_{idle}, a relative excess below $P_{\text{idle}}/(yP_a(u))$ for an allocation of y accelerators.

C. Placement Thresholds

By (59), the crossovers of access vs. edge and of edge vs. core are:

$$I^*_A \triangleq \frac{w_{EA}}{\varepsilon_A - \varepsilon_E} = \frac{w_{EA}}{(\text{PUE}_A - \text{PUE}_E)\,\kappa(a,m)}, \tag{60}$$

$$I^*_E \triangleq \frac{w_{CE}}{\varepsilon_E - \varepsilon_C} = \frac{w_{CE}}{(\text{PUE}_E - \text{PUE}_C)\,\kappa(a,m)}. \tag{61}$$

Each threshold, in FLOP/bit (i.e., J/bit divided by J/FLOP), is the value of I at which the extra computation power of the outer tier equals the transport power it avoids. For example, $I < I^*_E$ is equivalent to $R^{\text{flop}}(\varepsilon_E - \varepsilon_C) < R^{\text{bw}}w_{CE}$, so the edge beats the core exactly when it saves more transport energy than it spends in extra computation.

Theorem 1 (Optimal tier)**.** *Let a service with model m and computation-to-communication ratio I be served on GPU a. Whenever $I^*_A \le I^*_E$, the energy-optimal tier is*

$$t^\star = \begin{cases} \textit{access}, & I \le I^*_A, \\ \textit{edge}, & I^*_A \le I \le I^*_E, \\ \textit{core}, & I \ge I^*_E. \end{cases} \tag{62}$$

Proof. A tier is more efficient than another if serving the service there draws less power. From (59) and the definitions (60)–(61), the power differences between adjacent tiers are

$$P_E - P_A = R^{\text{bw}}\big[w_{EA} - (\varepsilon_A - \varepsilon_E)\,I\big] = R^{\text{bw}}(\varepsilon_A - \varepsilon_E)\,(I^*_A - I), \tag{63}$$

$$P_C - P_E = R^{\text{bw}}\big[w_{CE} - (\varepsilon_E - \varepsilon_C)\,I\big] = R^{\text{bw}}(\varepsilon_E - \varepsilon_C)\,(I^*_E - I). \tag{64}$$

Three cases arise.

Case $I \le I^*_A$. Then $P_A \le P_E$ and $P_A \le P_C$. This is because $I \le I^*_A$ makes (63) non-negative, so $P_A \le P_E$. Moreover, $I \le I^*_A \le I^*_E$ makes (64) non-negative, so $P_E \le P_C$, and hence $P_A \le P_E \le P_C$.

Case $I^*_A \le I \le I^*_E$. Then $P_E \le P_A$ and $P_E \le P_C$. This is because $I \ge I^*_A$ makes (63) non-positive, so $P_E \le P_A$, and $I \le I^*_E$ makes (64) non-negative, so $P_E \le P_C$.

Case $I \ge I^*_E$. Then $P_C \le P_E$ and $P_C \le P_A$. This is because $I \ge I^*_E$ makes (64) non-positive, so $P_C \le P_E$. Moreover, $I \ge I^*_E \ge I^*_A$ makes (63) non-positive, so $P_E \le P_A$, and hence $P_C \le P_E \le P_A$. □

Theorem 1 reduces the placement of an accelerator–model pair to a single comparison: the position of its ratio I relative to two thresholds. Fig. 5 illustrates the mechanism. Each tier is a line whose intercept is the transport energy per bit and whose slope is the computation energy per FLOP. Moving inward raises the intercept and lowers the slope, and the optimal tier is the lowest line at the service's I. Services with a low I are dominated by transport and are served at the access tier. Services with a high I are dominated by computation and are served at the core, where facility efficiency is best. The edge is optimal in between.

Corollary 1 (Dominated edge)**.** *If $I^*_A > I^*_E$, or equivalently $w_{EA}(\text{PUE}_E - \text{PUE}_C) > w_{CE}(\text{PUE}_A - \text{PUE}_E)$, the edge*

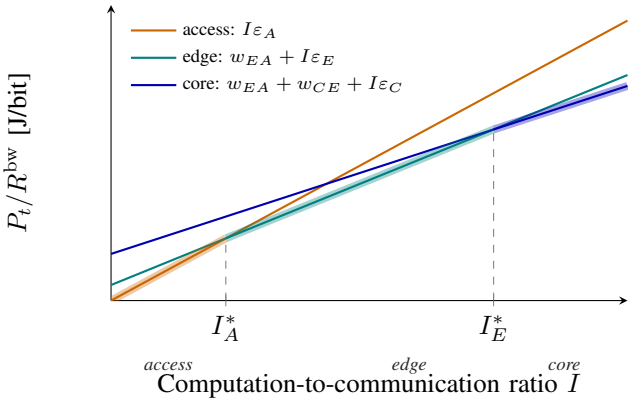


Fig. 5: Tier power per transported bit as a function of I (schematic). The optimal tier is the lower envelope (shaded), with crossovers at I_A^* and I_E^* (Theorem 1).

is never optimal, and the choice reduces to access vs. core. Their crossover is

$$I_{AC}^* \triangleq \frac{w_{EA} + w_{CE}}{\varepsilon_A - \varepsilon_C} = \frac{w_{EA} + w_{CE}}{(\mathrm{PUE}_A - \mathrm{PUE}_C)\,\kappa(a,m)}, \quad (65)$$

and the energy-optimal tier is

$$t^\star = \begin{cases} \textit{access}, & I \le I_{AC}^*, \\ \textit{core}, & I \ge I_{AC}^*. \end{cases} \quad (66)$$

Proof. The equivalence follows from (60)–(61), since $\kappa(a,m)$, common to both thresholds, cancels. For any I, either $I < I_A^*$, so $P_A < P_E$ by (63), or $I \ge I_A^* > I_E^*$, so $P_C \le P_E$ by (64). Hence the edge is never optimal. Finally, from (59),

$$\begin{aligned} P_C - P_A &= R^{\mathrm{bw}}\big[w_{EA} + w_{CE} - (\varepsilon_A - \varepsilon_C)\,I\big] \\ &= R^{\mathrm{bw}}(\varepsilon_A - \varepsilon_C)\,(I_{AC}^* - I), \end{aligned} \quad (67)$$

so $P_A \le P_C$ if and only if $I \le I_{AC}^*$. □

Corollary 2 (Compute-bound workloads)**.** *If $\sigma(m) \ge \sigma^*(a,m)$, then $\hat{c}(a,m) = c^{\mathrm{flop}}u$ and*

$$\kappa_{\mathrm{cb}} \triangleq (1+\alpha)\,\varphi\,\frac{P_a(u)}{c^{\mathrm{flop}}u}. \quad (68)$$

It follows that the thresholds I_A^*, I_E^* and I_{AC}^* obtained from (60)*,* (61) *and* (65) *with $\kappa = \kappa_{\mathrm{cb}}$ are independent of $\sigma(m)$ and of M_{model}. The placement is then a function of I alone.*

Corollary 3 (Memory-bound workloads)**.** *If $\sigma(m) < \sigma^*(a,m)$, then $\hat{c}(a,m) = (c^{\mathrm{vram}} - M_{\mathrm{model}})\,\sigma(m) < c^{\mathrm{flop}}u$ and*

$$\kappa_{\mathrm{mb}} \triangleq (1+\alpha)\,\varphi\left[\frac{P_{\mathrm{idle}}}{(c^{\mathrm{vram}} - M_{\mathrm{model}})\,\sigma(m)} + \frac{P_{\mathrm{TDP}} - P_{\mathrm{idle}}}{c^{\mathrm{flop}}}\right] > \kappa_{\mathrm{cb}}. \quad (69)$$

It follows that the thresholds I_A^*, I_E^* and I_{AC}^* obtained from (60)*,* (61) *and* (65) *with $\kappa = \kappa_{\mathrm{mb}}$ are lower than in the compute-bound case, and decrease further as $\sigma(m)$ decreases. The placement then depends on $\sigma(m)$ and M_{model} as well as on I.*

Proof. Since $P_a(u) = P_{\mathrm{idle}} + u\,(P_{\mathrm{TDP}} - P_{\mathrm{idle}})$, (68) can be written as $\kappa_{\mathrm{cb}} = (1+\alpha)\varphi\,[P_{\mathrm{idle}}/(c^{\mathrm{flop}}u) + (P_{\mathrm{TDP}} - P_{\mathrm{idle}})/c^{\mathrm{flop}}]$. Subtracting it from (69),

$$\kappa_{\mathrm{mb}} - \kappa_{\mathrm{cb}} = (1+\alpha)\,\varphi\,P_{\mathrm{idle}}\left[\frac{1}{\hat{c}(a,m)} - \frac{1}{c^{\mathrm{flop}}u}\right] > 0,$$

because $\hat{c}(a,m) < c^{\mathrm{flop}}u$. Every threshold is inversely proportional to κ, so it is lower with κ_{mb}. Since κ_{mb} grows as $\sigma(m)$ decreases, the thresholds decrease with $\sigma(m)$. □

D. Infrastructure and Workload Levers

Theorem 1 and Corollaries 2–3 identify the mechanisms that shift a service across tiers.

Lever 1 – Reduce the computation-to-communication ratio I. Decreasing the arithmetic work per communicated bit moves a workload left on the I axis and toward the access tier. This explains the taxonomy of Sec. V-E. Discriminative analytics process high-rate streams with lightweight models and occupy the low-I region. Generative models perform orders of magnitude more computation per transmitted bit and fall in the region where centralized execution is most energy efficient.

Lever 2 – Reduce the facility-efficiency gap. By (61), $I_E^* \to \infty$ as $\mathrm{PUE}_E \to \mathrm{PUE}_C$. In this limit no workload prefers the core over the edge, while the split between access and edge remains governed by I_A^*.

Lever 3 – Improve accelerator energy efficiency. Dividing the board power by a factor δ^{eff} divides $\kappa(a,m)$ by δ^{eff}. By (60), (61) and (65), this multiplies every threshold by δ^{eff} and moves services *outward*. Cheaper computation shrinks the absolute cost of the less efficient outer facilities relative to the transport they avoid.

Lever 4 – Relax the memory binding. For memory-bound services, increasing the available dynamic memory $c^{\mathrm{vram}} - M_{\mathrm{model}}$ lowers σ^* and raises $\hat{c}(a,m)$. This can be done with larger VRAM or with weight quantization, which reduces M_{model}. By Corollary 3, all thresholds move toward their compute-bound values, and the effect vanishes once $\sigma(m) \ge \sigma^*(a,m)$.

When the request source and response destination are geographically separated, ρ directly influences the preferred placement location. Workloads with $\rho \gg 1$ benefit from executing closer to the destination of the response, whereas workloads with $\rho \ll 1$ benefit from executing closer to the data source to minimize transport of the dominant traffic component.

VIII. Results

Think of having a subsection here describing the common network setting and services.... description of three tier network with one core node, XX edge nodes, and YY access nodes. [Note that even if we model them as single nodes with higher capacity, it is equivalent to multiple nodes per tier] and description of main services considered. This means you could move seciton a) and b) of infrastructure nature here.

In the different experiments, some of this variables will be changed to analyze ... [three tier topology], for all the

analysis of a single service, we assume that we have the same type of GPUs everywhere and that the number of gpu is not limiting [the type and number of GPUs is not a tier limiting factor, meaning that each tier has available, right, but for these analysis. Okay. Now, let's assume that]

A. Reference Setting

All results in this section are obtained on a common network and service setting, described once here. Any deviation from it is stated explicitly where it occurs.

1) Network:

We consider a single three-tier segment, *core* ↔ *edge* ↔ *access*, with the users attached at the access tier. The segment comprises one core site, one edge site and 25 access sites, the latter being the radio sites whose traffic the edge↔access link aggregates.

Each tier is represented by a *single* node carrying the summed capacity of its sites. This is not a loss of generality: the model charges accelerator count, traversed links and facility overhead, none of which depends on which co-tier site a replica lands on, so a tier modelled as one node of capacity C is equivalent to k co-tier nodes summing to C.[8] The results below should therefore be read per segment, and scale by replicating the segment.

The three tiers differ in two ways that pull in opposite directions, and the placement decision arbitrates between them. Facility efficiency improves *inward*: following Table I we take $\mathrm{PUE}_u = 1.2$, 1.3 and 1.8 at core, edge and access, with a common over-provisioning $\varphi_u = 1.1$. Transport cost, in contrast, grows inward: serving at the access node requires no inter-tier transport, serving at the edge crosses one segment, and serving at the core crosses two. The corresponding energy coefficients of (12) are $e_{uv} = 4.70$ and 2.35 nJ/bit on the edge↔core and access↔edge links, consistent with the per-segment range of Table II and the hop counts n_{uv} therein, and the one-way propagation delays are $\tau_{uv} = 25$ ms and 8 ms respectively.

2) Compute infrastructure:

Throughout the single-service analyses of this section we assume that *neither the type nor the number of accelerators is a tier-limiting factor*: every tier is offered the full catalogue of Table VII, in a quantity that never binds. No service is therefore restricted by which hardware a tier happens to hold, nor by how much of it, and every placement reported below is attributable to the service's own compute, memory and bandwidth demand and to the tier's facility efficiency and distance. Sec. VIII-D states explicitly where this assumption is relaxed.

Table VII gives the per-accelerator values used for the symbols introduced in Table I. For the remaining parameters we adopt a single value inside the ranges already tabulated there: utilization $u^{\mathrm{flop}}_{a,m} = 0.70$, idle draw $P_{\mathrm{idle},a} = 0.20\,P_{\mathrm{TDP},a}$, and host-node overhead $\alpha_p = 0.21$. No power model is introduced here: site power is (34) and its per-model accelerator term the load-proportional form already stated in the problem formulation, both evaluated at the values above, with link energy from (12).[9]

TABLE VII: Accelerator classes offered at every tier, and the values adopted for the parameters of Table I.

Accelerator a	c^{flop}_a [PFLOP/s]	c^{vram}_a [GB]	$P_{\mathrm{TDP},a}$ [W]	$c^{\mathrm{flop}}_a / P_{\mathrm{TDP},a}$ [TFLOP/s/W]
H100	4.00	80	700	5.71
L40S	0.91	48	350	2.60
A10G	0.50	24	150	3.33
T4	0.13	16	70	1.86

$u^{\mathrm{flop}}_{a,m} = 0.70$ $\quad P_{\mathrm{idle},a} = 0.20\,P_{\mathrm{TDP},a}$ $\quad \alpha_p = 0.21$ $\quad \varphi_u = 1.1$
$\mathrm{PUE}_u = 1.2\,/\,1.3\,/\,1.8$ (core / edge / access)

One modelling choice specific to the evaluation does need stating, because it is not implied by the formulation: the number of active accelerators y^m_p is restricted to the integers. An accelerator is therefore either energized — and charged its idle draw, however lightly it is loaded — or not present at all. This is what makes accelerator *count*, and not utilization alone, an energy-relevant quantity, and it is the origin of the step-wise behaviour visible in several of the sweeps below.

Two properties of this catalogue determine what the optimiser does with that freedom. First, the accelerators are not equally efficient: the last column of Table VII spans a factor of three, and the H100 leads it. Second, an accelerator can only host a model whose weights fit its memory, c^{vram}_a, so the larger models are eligible on fewer classes. Given both, the optimiser selects the H100 at every tier for every service throughout Sections VIII-C and VIII-D. The resulting deployment is thus *homogeneous by choice rather than by assumption* — the same accelerator type everywhere — which is why we refer to "the accelerator" in the singular below, and why any difference between tiers in what follows comes from facility efficiency and distance alone, never from hardware.

3) Services:

We deploy the seven services of Table VIII: T2T (Llama-3-70B), T2I (SD 3.5 Large), T2V (Wan2.1-14B), YOLO (YOLOv8n) and AR (ViT-L/14).[10] Unless stated otherwise each is solved on its own instance of the topology, so that a reported curve characterises that service rather than a shared deployment.

4) What varies across the experiments:

The setting above is the common baseline; each experiment then varies a different subset of it, holding the rest fixed, so that every reported effect is attributable to the variable being swept. Sec. VIII-B keeps the baseline unchanged and compares

[8] The equivalence is exact for the energy and delay terms; it neglects only the per-site integer rounding of the accelerator pool, which would add at most one partially-loaded accelerator per site.

[9] The two per-model power constraints in the formulation (the utilization form and the load form) currently share the label `eq:c-pgpu`, so it cannot be cited unambiguously; giving them distinct labels would let this sentence point at the load form directly.

[10] Naming: this section labels services by *model* name. Table VIII labels them by *class* in its first column, where `VID` is the class of the two YOLOv8 rows and `AR` that of the ViT-L/14 row; `tab:workload` and the caption of `tab:perreq` follow the same convention. Decide before submission whether the results text should switch to the class labels or the tables to the model names – at present the figures use model names.

services at a single operating point. Sec. VIII-C varies the *infrastructure*: first the watts-per-FLOP of the accelerator and the watts-per-bit of the links (δ^{eff}, δ^{bw}), then the memory and throughput of the accelerator itself (δ^{vram}, δ^{flops}); the delay budget is held relaxed there so that latency never vetoes a tier. Sec. VIII-D does the converse: it holds the infrastructure at the baseline and varies the end-to-end delay budget, which is also the only place where a tier becomes unavailable to a service. Two assumptions are relaxed in exactly one place each, and are flagged where they are: the single-accelerator-class sweep of Sec. VIII-C3, and the reachability restrictions induced by the delay budget in Sec. VIII-D.

B. Impact of Service Nature

Fig. 6a locates the evaluated services on the I-axis and reports the total power of the energy-optimal deployment of each. Assuming that the deployed GPU provides sufficient memory capacity to sustain its target compute utilization for the considered service, i.e., memory does not become the limiting resource before the desired utilization level is reached, by Corollary 2 the thresholds of Theorem 1 reduce to the constants $I^*_{A,\mathrm{cb}} = w_{EA}/\big((\mathrm{PUE}_A - \mathrm{PUE}_E)\,\kappa_{\mathrm{cb}}\big)$ and $I^*_{E,\mathrm{cb}} = w_{CE}/\big((\mathrm{PUE}_E - \mathrm{PUE}_C)\,\kappa_{\mathrm{cb}}\big)$, with κ_{cb} given by (68). Placement is then a function of the abscissa alone: a service is assigned to a tier by its position on the I axis, and the shaded bands may be read directly as a decision rule.

For memory-bound services ($\sigma < \sigma^*$), the effective throughput $\hat{c}(\sigma)$ of Lemma 1 falls below $c^{\mathrm{flop}}u$. The one-dimensional spectrum of Fig. 6a then becomes the two-dimensional map of Fig. 7, whose boundaries follow Corollary 3. They are vertical above σ^*, and below it they bend toward lower I as increasing, concave functions of σ given by (69). By Corollary 1 the two boundaries remain translates of each other in $\log I$, so the edge band keeps a constant logarithmic width.

Fig. 8 isolates the effect of the compute-to-transport intensity by sweeping a synthetic service along the I-axis at constant processing demand, varying $I = F/B$ through decreasing B.

Panel (a) reproduces the three-region structure of Theorem 1. In the access region the total power is independent of I, as it must be: access execution avoids both transport segments, so $E_{\mathrm{access}} = F\varepsilon_A(\sigma)$ carries no dependence on B. As I increase (and B decrease), beyond $I^\star_A$ the optimum moves to the edge and the total decreases monotonically as the residual payload shrinks, and beyond $I^\star_E$ it moves to the core. The total is continuous at both crossovers, since by construction the two tiers are of equal cost there.

Panel (b) separates the two terms and exhibits the mechanism. The compute component is piecewise constant, stepping down at each crossover by the facility ratio $\varepsilon_t/\varepsilon_{t'}$ of (R2), while the transport component is zero at the access tier, jumps to Bw_{EA} when execution moves to the edge, and jumps again by the factor $1 + w_{CE}/w_{EA}$ when it moves to the core.

Panel (c) confirms that the accelerator allocation is invariant along the sweep. Because σ is constant, Lemma 1 gives a constant effective throughput $\hat{c}(\sigma)$ and hence a constant block count, which migrates between tiers at the crossovers but never changes magnitude.

Panel (d) reports the resulting transport load, which decays as $1/\mathcal{I}$ on each active segment and is identically zero while the service is served at the access tier.

C. Impact of Infrastructure Nature

Sec. VII argued that an infrastructure improvement can displace a service's energy-optimal placement, either by changing the relative cost of computing at each tier or by better matching accelerator memory to the workload's footprint. We now evaluate both effects on the full MILP. Unlike the analytical spectrum of Sec. VIII-B, which reads placement off the I axis under a compute-bound assumption, the MILP imposes no such assumption: memory is free to become the binding resource, and for one of the five services it does. To isolate them from the delay veto studied separately in Sec. VIII-D, every experiment in this subsection is solved at a relaxed delay budget of 150 ms, at which all three tiers are reachable for all five services; any placement change reported here is therefore attributable to the infrastructure lever alone.

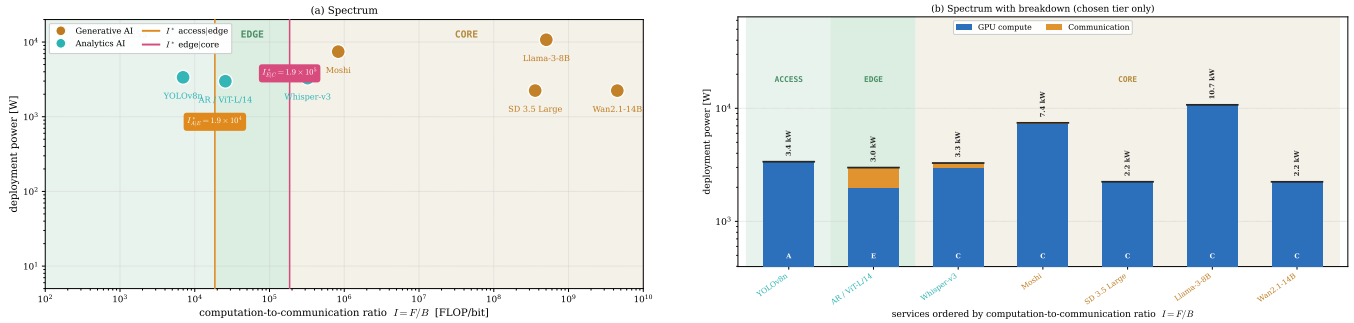
a) Efficiency levers: We model a future accelerator or transport generation as a multiplicative factor applied uniformly to the deployed fleet, taking the reference setting of Sec. VIII-A as the $1\times$ baseline. $\delta^{\mathrm{eff}} \geq 1$ divides the board power of every accelerator, improving watts-per-FLOP without changing throughput; $\delta^{\mathrm{bw}} \geq 1$ divides the watts-per-Mbps of every link; $\delta^{\mathrm{vram}} \geq 1$ multiplies per-chip VRAM; and $\delta^{\mathrm{flops}} \geq 1$ multiplies raw throughput *and* board power together, representing a physically larger chip rather than a more efficient one. All four are dimensionless multipliers, not absolute FLOP/W or W/Mbps values.

1) Compute and communication efficiency: We sweep δ^{eff} from $1\times$ to $5\times$ and δ^{bw} from $1\times$ to $4\times$ on a 41×31 grid. We report three services, chosen so that they occupy the three placement regions that Sec. VIII-B establishes along the I axis: T2I sits far above I^*_E and is served at the core, YOLO lies below I^*_A and is served at the access tier, and AR falls between the two thresholds and is served at the edge. The question here is what happens to each of them when the infrastructure, rather than the service, is what changes.

Fig. 9 must be read as *pairs of panels*, one column per service. The top row gives the total-power reduction over the plane; the bottom row gives the tier the optimizer selects at each point, with bar height equal to the aggregate power actually paid. The top row alone is ambiguous: two services can produce equally flat contours for opposite reasons, and only the bottom row separates them. T2I and YOLO are precisely such a pair — nearly indistinguishable above, and opposite in nature below.

T2I is the pure compute case. Its iso-reduction contours are exactly horizontal: $\delta^{\mathrm{eff}} = 5$ yields an 80.0% reduction, while δ^{bw} contributes at most 0.02% anywhere on the grid. The bottom panel explains why: the service sits at the core over the entire plane, and transport accounts for 0.01% of its power there. Its contours are flat because it has almost no transport bill to reduce.

YOLO's panel is flat for the opposite reason. Its 1.01 Tbps uplink is so dominant that the optimizer places the service at



(a) Spectrum.

(b) Spectrum with breakdown.

Fig. 6: Compute-bound regime ($\sigma \geq \sigma^*$, Corollary 2): the placement decision is driven only by the computation-to-communication ratio I. (a) Services on the I axis with the thresholds I_A^* and I_E^*. (b) Power of the chosen tier, split into GPU compute and communication.

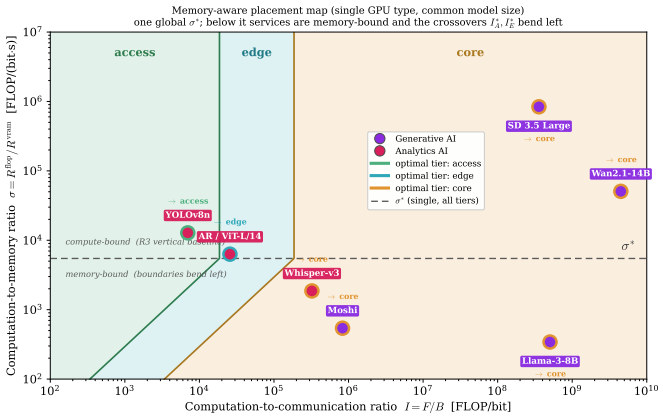


Fig. 7: Memory-aware placement map under (H), common model footprint. Above σ^* the tier boundaries are vertical (Corollary 2). Below σ^* both boundaries bend toward lower I according to (69) (Corollary 3), and they remain translates of each other in $\log I$ (Corollary 1). this plot is assuming the same available dynamic memory capacity

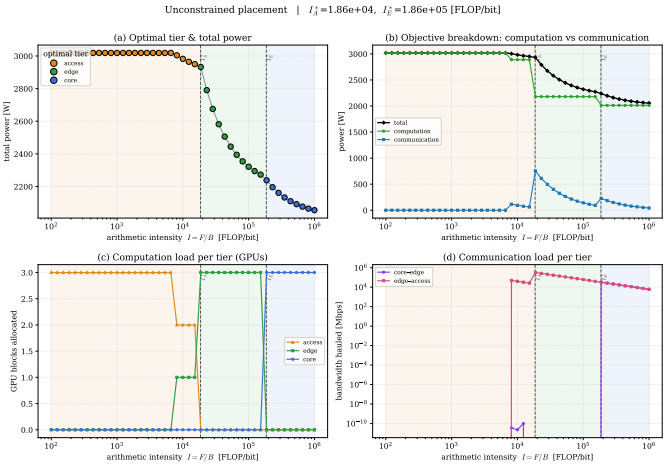


Fig. 8: Synthetic compute-bound service swept along the I axis at constant processing demand (Corollary 2): (a) optimal tier and total power; (b) computation and communication components; (c) GPUs allocated per tier; (d) bandwidth hauled per segment.

the access tier — the point closest to the sources within the optimized segment — and absorbs the 1.8 facility overhead there rather than carry that stream further inward. The inter-tier transport it then pays is zero, which is a statement about the modelled segment and not about the service: the radio hop between the devices and the access node still carries the stream, but it lies upstream of the ingress at which our optimization begins. Because the service has already reached the innermost[11] placement the model offers, δ^{bw} has no further relocation to buy, and it is inert over 1,227 of the 1,271 feasible grid points (97%). It acts only along the $\delta^{\mathrm{eff}} = 1$ edge, where transport becomes cheap enough ($\delta^{\mathrm{bw}} \gtrsim 2.5$) for a move to the edge to pay: total power falls from 3.370 to 3.027 kW (-10.2%), the compute term dropping from 3.370 to 2.434 kW as the better facility takes over, while a 0.593 kW transport term appears for the first time. The saving is therefore a relocation effect rather than a reduction of an existing transport bill.

AR, in contrast, genuinely pays for transport: 20.8% of its 2.989 kW baseline is communication, and it sits at the edge. Here δ^{bw} delivers a smooth, graded saving with no change of tier, shrinking the transport term from 0.621 to 0.155 kW and total power by 15.6%. Its contours consequently bend, whereas YOLO's remain flat over almost the whole plane. The two levers also act on AR in opposite directions: raising δ^{eff} makes compute cheap enough that the access tier's 38% PUE premium over the edge becomes smaller in absolute terms than the transport it avoids, so AR migrates edge → access already at $\delta^{\mathrm{eff}} \approx 2$; raising δ^{bw} removes the reason to be close to the source and holds it at the edge. The interaction is visible as a non-monotonic sensitivity: the saving obtainable from δ^{bw} alone peaks at 18.3% for $\delta^{\mathrm{eff}} = 2$ and decays to 4.1% at $\delta^{\mathrm{eff}} = 5$, because by then the service has already relocated to a tier where communication is free.

[11] Innermost with respect to the sources, i.e. the tier at which the fewest optimized links are traversed.

Two conclusions follow, and the distinction between them matters. The first is about *mechanism*: a transport improvement is banked by a communication-dominated service in one of two ways. AR, whose transport bill is large but not overwhelming, converts δ^{bw} directly into energy, smoothly and without moving. YOLO converts it into *placement*: a single migration step, after which the response saturates because no tier closer to the sources exists inside the optimized segment. Communication efficiency is therefore not a single lever with a single effect; what it buys depends on whether the service still has somewhere to go.

The second conclusion is a caveat on the first. YOLO's flat response is bounded by our modelling scope, not by the physics of the service: the access-to-device segment is not part of the optimized topology, and an efficiency improvement there would act on precisely the traffic that pins the service to the access tier. The experiment therefore establishes that, *within a cloud-edge-access segment*, an extreme uplink saturates the placement lever — not that such a service is insensitive to communication efficiency in general. Extending the model past the access ingress is a natural next step, and is where the remaining sensitivity of this class of service would appear.

2) Where the saving is taken: Fig. 10 decomposes the two levers separately, sweeping one at a time and reporting absolute power split by tier and by communication, so that a change in bar height (how much energy) can be separated from a change in bar composition (where it is spent).

For T2I the two sweeps could hardly be more different in information content and more similar in shape: under δ^{eff} the bar is 100% core at every point and falls monotonically from 2.240 to 0.448 kW, while under δ^{bw} it is flat to four significant figures. The service is a single-lever workload, and its composition never moves.

YOLO shows the opposite decomposition. Its δ^{eff} sweep is 100% access at every point, falling from 3.370 to 0.674 kW with no communication segment ever appearing: the service is uplink-dominated yet its energy bill is entirely compute, precisely because placement has eliminated the transport term. Its δ^{bw} sweep is where the composition changes: the bar stays pure access and perfectly flat until $\delta^{\mathrm{bw}} \approx 2.5$, at which point a split edge/access allocation appears (17.8% edge, 76.4% access, 5.8% communication), resolving by $\delta^{\mathrm{bw}} = 4$ into an edge-only deployment of 80.4% compute and 19.6% communication. Total power falls by only 10.2% across the entire sweep — a small energy gain purchased by a complete change of placement.

AR closes the picture by exhibiting each behaviour on a different axis. Under δ^{eff} it starts at the edge with a visible 20.8% communication segment and then, from $\delta^{\mathrm{eff}} = 2$ onward, jumps to a 100% access composition in which the communication segment vanishes entirely; total power falls from 2.989 to 0.656 kW. Under δ^{bw} it never moves: the tier composition is edge throughout and only the communication share shrinks, $20.8\% \to 13.4\% \to 9.5\% \to 6.1\%$, for a 15.6% total reduction. Compute efficiency changes *where* this service runs; communication efficiency changes only *how much* it costs where it already is. For an operator this distinction matters: the first lever alters the infrastructure footprint and the placement policy, the second is a pure operating-expenditure improvement.

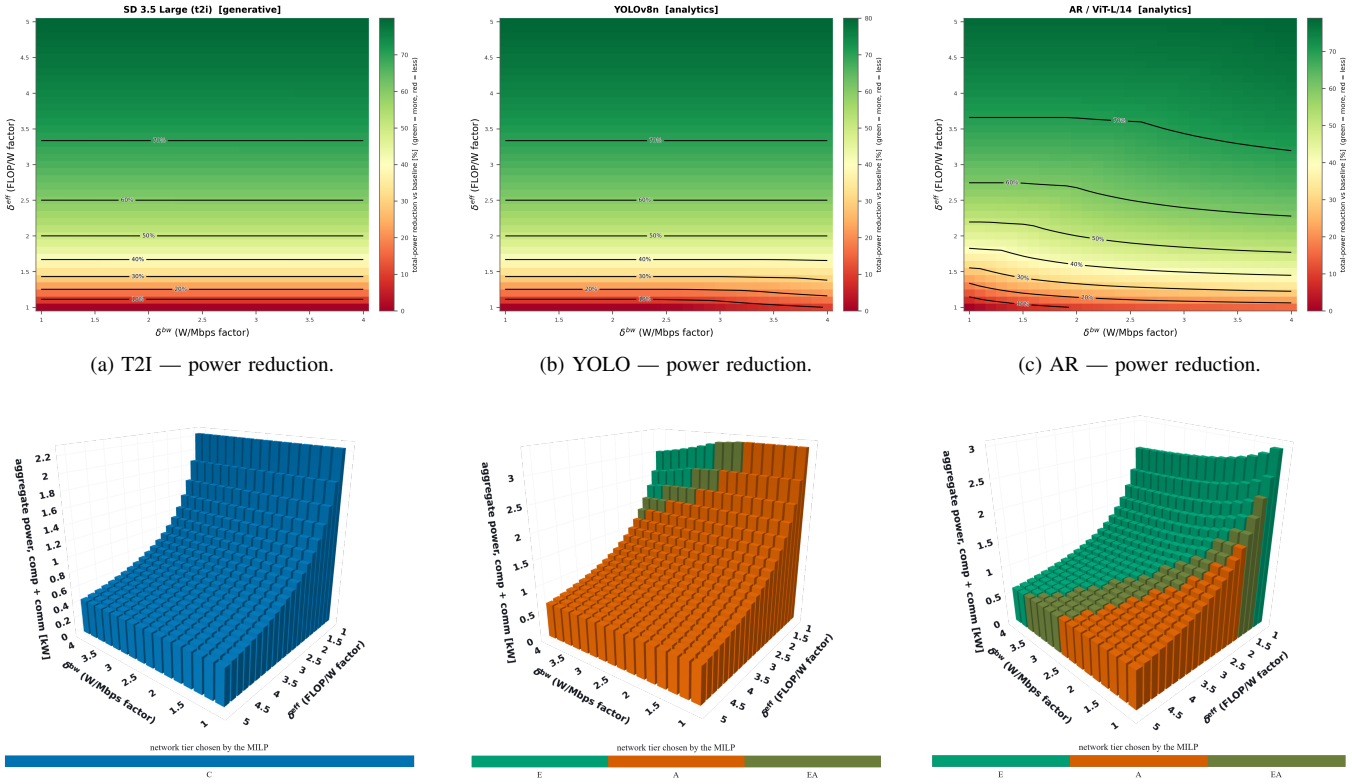
3) Memory/FLOP configuration: A chip's effective throughput is limited by whichever resource saturates first (Sec. VII). We therefore sweep δ^{vram} from $1\times$ to $4\times$ against δ^{flops} from $1\times$ to $5\times$, with $\delta^{\mathrm{eff}} = 1$, and report all five services in Fig. 11. Bar height is the optimal chip count and colour is $\rho = P/P_{\mathrm{today}}$. Because δ^{flops} scales board power together with throughput, this plane deliberately contains no efficiency improvement: it answers whether a bigger accelerator, as opposed to a better one, is worth building for each workload. Unlike the rest of this section, the sweep scales *one* accelerator class rather than selecting from the catalogue of Table VII: the question here is what the next generation of a given part buys, so holding the class fixed and scaling its three defining parameters is the meaningful experiment.

The answer separates the five services into exactly two groups, and the split is the empirical counterpart of the σ^* boundary of Sec. VIII-B: T2T is the only service that falls on the memory-bound side of it, and the only one for which the plane is informative in the δ^{vram} direction. At $\delta^{\mathrm{vram}} = 1$ its chip count is pinned at 41 — set by its 2.56 TB of aggregate KV cache against the 63.9 GB of headroom each chip retains once the 16.1 GB of weights are resident — for every value of δ^{flops}; adding raw throughput buys nothing while still costing power, so ρ rises linearly to $4.42\times$ *worse* than today at $\delta^{\mathrm{flops}} = 5$. Along the memory axis the same service contracts steadily: 41 chips at $1\times$, 31 at $1.25\times$, 25 at $1.5\times$, 18 at $2\times$ and 9 at $4\times$, with power falling from 10.73 kW to $\rho = 0.33$.

The *shape* of that response is itself informative, and it is governed by how much of the chip the model weights occupy. Here the weights take only 20% of an H100, so headroom grows almost in proportion to δ^{vram} and the chip count falls accordingly — a 25% VRAM increase removes 24% of the deployment. A weight-dominated deployment behaves very differently: served at reduced precision, Llama-3-70B occupies 72 of the same 80 GB and leaves only 8 GB for cache, so the same 25% VRAM increase more than triples the usable headroom and removes two thirds of the chips at once. The memory lever is therefore not a fixed-gain knob: its leverage is set by the ratio of weights to cache on the chip, and it is sharpest exactly where the weights crowd the cache out.

T2I, T2V and YOLO form the opposite group: their planes are completely flat along δ^{vram}, since none is ever memory-bound in this range, and their chip count falls with δ^{flops} alone, from 3 to 2 to 1. Notably ρ never drops below 1.00 and oscillates between 1.00 and 1.20: because chip counts are integer and small, a doubling of throughput does not halve the count, and the residual chip is now a higher-power part. AR is a marginal intermediate case — its 134 GB of inference memory needs two chips at $\delta^{\mathrm{vram}} \leq 1.5$ but only one from $\delta^{\mathrm{vram}} = 2$, which caps ρ at 1.16 instead of 1.57 — but the effect is a single-chip step, not the order-of-magnitude collapse seen for T2T.

Two conclusions follow. First, VRAM growth is a paradigm-



(a) T2I — power reduction. (b) YOLO — power reduction. (c) AR — power reduction.

(d) T2I — tier and aggregate power. (e) YOLO — tier and aggregate power. (f) AR — tier and aggregate power.

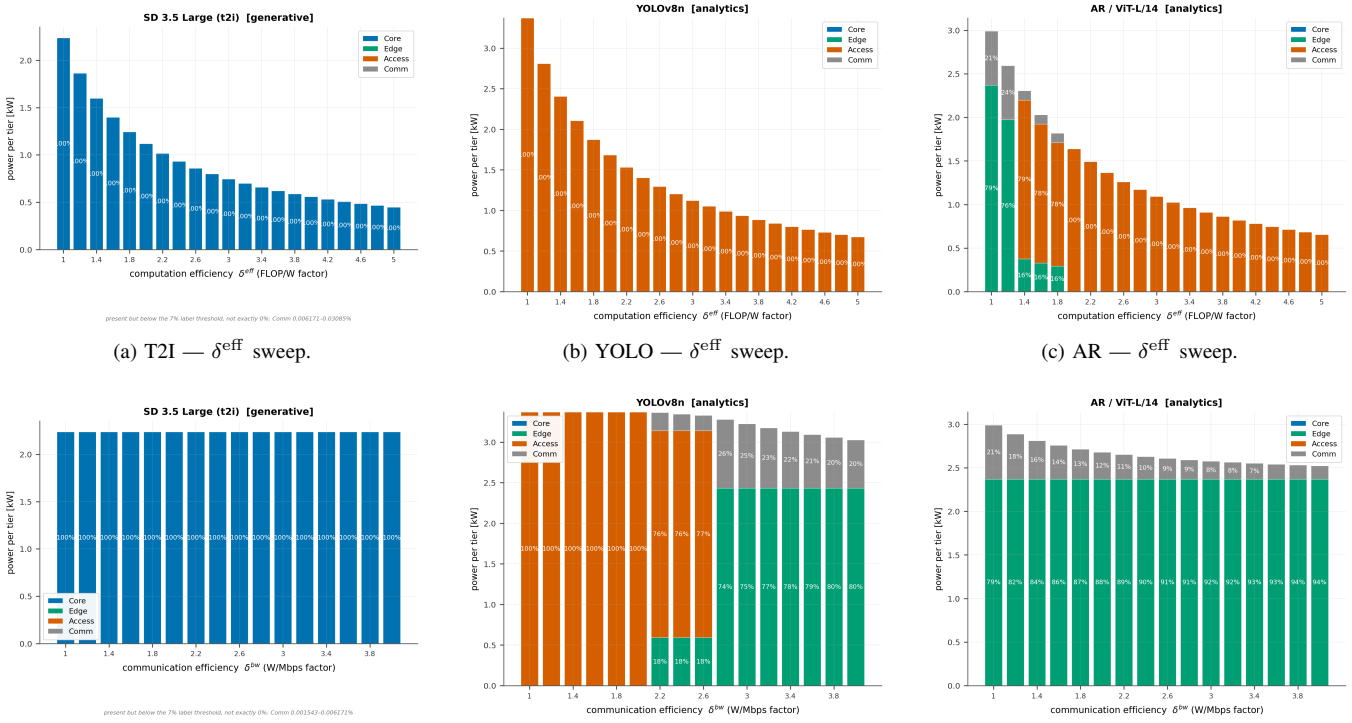
Fig. 9: Efficiency coupling over the $(\delta^{\text{eff}}, \delta^{\text{bw}})$ plane at relaxed QoS. Top row: total-power reduction relative to the $(1, 1)$ baseline; horizontal contours indicate a service driven by δ^{eff} alone, curved contours a joint dependence. Bottom row: the same grid with bar height equal to aggregate power (computation plus communication) and colour equal to the selected tier.

specific lever rather than a general one: it pays only where the KV cache is what genuinely exhausts the chip, which among these workloads means the autoregressive service alone. Second, raw throughput growth without a matching efficiency gain is not an energy improvement at all — for four of the five services it leaves power unchanged or slightly worse, and for the memory-bound one it is strictly harmful. Only δ^{eff}, which improves watts-per-FLOP directly, reduces power for every workload.

4) Facility efficiency and coexistence: The levers examined so far change what is deployed — a better accelerator, a bigger one, a cheaper link. A third infrastructure parameter changes none of them: the facility overhead PUE_u multiplies the power of whatever sits at site u without altering the hardware, the workload or the topology. It is also the parameter an operator is least free to assign per service, since it is a property of the building rather than of the deployment. We therefore sweep it jointly over the edge and access tiers, with the core pinned at 1.3 — the top of its own range in Table I, and hence the case least favourable to the core, where its inward efficiency advantage is smallest. Edge and access are swept from that same value up to their own cited ceilings (1.50 and 2.00 respectively), and the model's ordering assumption $\text{PUE}_{\text{core}} \leq \text{PUE}_{\text{edge}} \leq \text{PUE}_{\text{access}}$ admits 101 of the 121 grid points; the remainder (grey in Fig. 12) is excluded by that assumption rather than left unsolved.

The plane separates the five services into the same three regimes the earlier experiments produced, and for the same reason. The three generative services are flat in *both* directions: T2T holds 11.44 kW, T2I 2.43 kW and T2V 2.42 kW over the entire admissible region, a 0.0% spread. They occupy the core in 90 of the 101 cells, and the apparent edge band along $\text{PUE}_{\text{edge}} = 1.30$ is a tie rather than a migration: at that line all three facilities are equally efficient and the power is unchanged to four significant figures. YOLO is the opposite extreme. It is served at the access tier in *every* admissible cell, so it pays $\text{PUE}_{\text{access}}$ in full and its contours are horizontal: power rises from 2.43 to 3.74 kW, a 53.8% swing produced by the access facility alone. AR is once again the informative intermediate, and the only service with a genuinely two-dimensional response: it is served at the edge in 42 cells, at the access tier in 43, and splits between them in the remaining 16, tracing the boundary along which the two facilities trade against the transport that separates them.

The reading is that exposure to a tier's facility efficiency is decided by placement, and placement is decided by the service's own intensity I. A service is insensitive to the efficiency of every tier it does not use, which is why the core-resident generative workloads ignore both axes entirely. More pointedly, the exposure is largest precisely where the service has no way to escape it: YOLO is pinned to the access tier by its uplink, so the building it sits in is its dominant



(a) T2I — δ^{eff} sweep. (b) YOLO — δ^{eff} sweep. (c) AR — δ^{eff} sweep.

(d) T2I — δ^{bw} sweep. (e) YOLO — δ^{bw} sweep. (f) AR — δ^{bw} sweep.

Fig. 10: Per-tier energy composition under the single-lever sweeps, relaxed QoS. Bar height is absolute power; each segment is a tier's (or communication's) contribution, labelled in-bar with its share of the total.

energy parameter — a 53.8% swing, against the 10.2% that the whole δ^{bw} range could buy it (Sec. VIII-C1). For this class of service, facility efficiency is not a secondary consideration behind transport efficiency; it is the primary one.

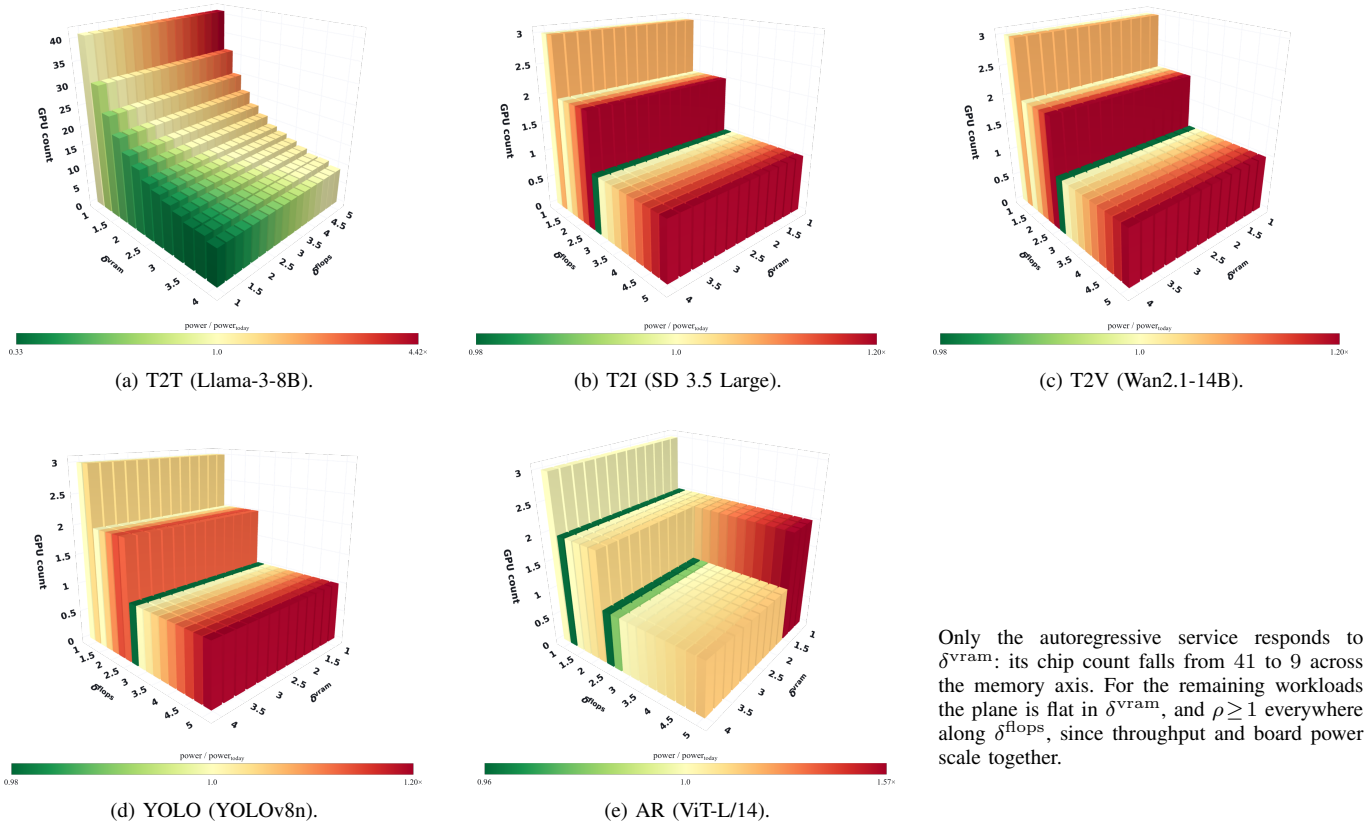
Fig. 13 closes the subsection by relaxing the one simplification kept throughout: that each service owns its own copy of the infrastructure. With all five sharing a single estate the objective still sums over models, so the per-service split is exact rather than apportioned. Moving from a full accelerator catalogue at every tier to the realistic estate — full catalogue at the core, A10G/T4 at the edge, T4 only at the access tier — raises total power from 21.40 to 25.46 kW (+19.0%), and the entire 4.06 kW penalty is borne by two services. T2T, T2I and T2V are unchanged to the kilowatt, because the tier they occupy is the one whose catalogue is complete. YOLO rises from 3.37 to 6.38 kW (+89%) and AR from 2.99 to 4.04 kW (+35%), and the increase is largely transport: evicted from the T4-only access tier, YOLO must serve from the edge and communication becomes 37% of its bill, while AR splits across core and edge at 41%. Catalogue breadth in the outer tiers is therefore not a uniform infrastructure property but a per-service one, and its value accrues entirely to the services that the placement analysis sends there.

D. Impact of QoS

Sec. VII treats the end-to-end delay budget as a veto rather than as a second placement objective: it removes infeasible tiers from consideration, and energy alone selects among the survivors. We now test that claim on the full MILP with the latency constraints active, on the same infrastructure and workloads as above.

The budget a service must meet is the sum of four terms: the operator's time-to-first-response (TTFR), the radio round trip to the access node, the per-hop transmission time of the request and response commodities, and twice the propagation delay of every inter-tier segment traversed. TTFR is the interval between the request arriving at the serving accelerator and the first unit of output leaving it — the first token for T2T, the first decoded pixel or frame for T2I and T2V, the first detection for YOLO, the first rendered overlay for AR — and is fixed to a common 25 ms floor so that the sweep is not dominated by paradigm-specific processing time and the budget is free to bind on transport instead. With the transport parameters of Sec. VIII-C, this places the access tier within reach at 36.5–38 ms for the three generative services and at 46 ms for YOLO and AR, whose 10 ms uplink transmission time reflects their continuous capture streams; the edge becomes reachable at 54–73 ms and the core at 105.5–134 ms. Fig. 14 sweeps the budget from 30 to 150 ms.

The three generative services (Fig. 14(a)–(c)) follow an identical, veto-driven pattern. All are infeasible at 30 ms, become feasible at 40 ms with an access-tier placement, and then migrate access → edge → core as the budget relaxes, reaching a core-only deployment at 115 ms and remaining there. Total power falls monotonically along the way, by 29.5% for T2T (15.22 → 10.73 kW) and by 33.3% for both



(a) T2T (Llama-3-8B).

(b) T2I (SD 3.5 Large).

(c) T2V (Wan2.1-14B).

(d) YOLO (YOLOv8n).

(e) AR (ViT-L/14).

Only the autoregressive service responds to δ^{vram}: its chip count falls from 41 to 9 across the memory axis. For the remaining workloads the plane is flat in δ^{vram}, and $\rho \geq 1$ everywhere along δ^{flops}, since throughput and board power scale together.

Fig. 11: Hardware-scaling coupling in the $\delta^{\text{vram}} \times \delta^{\text{flops}}$ plane ($\delta^{\text{eff}} = 1$), relaxed QoS. Bar height is the optimal chip count; colour is $\rho = P/P_{\text{today}}$ (green: savings, red: worse than today).

T2I and T2V ($3.36 \rightarrow 2.24$ kW). Since these services carry essentially no communication energy, the entire reduction is a facility-efficiency effect: the core is their energy-optimal tier throughout, and a tight deadline does not change that preference — it merely forbids it, forcing the workload outward onto less efficient facilities. The intermediate budgets are served by mixed allocations (e.g. T2T at 48 ms splits 1.7% core, 56.3% edge, 42.0% access), which is the optimizer buying as much core capacity as the deadline allows.

YOLO (Fig. 14(d)) is the exact opposite regime. It is infeasible below 48 ms, and from there its bar is a single uniform access-tier segment of 3.370 kW at *every* budget up to 150 ms: total variation 0.0%. Its energy-optimal tier is also the nearest one, so once the veto is lifted there is nothing for the relaxation to buy. Importantly, the core is reachable for YOLO from 134 ms onward and the service still does not move: the choice is energy-driven, not latency-driven.

AR (Fig. 14(e)) is again the informative intermediate. It shares YOLO's 48 ms feasibility threshold, but unlike YOLO it does improve with relaxation — from 3.280 kW to 2.989 kW, a modest 8.9% — migrating access $\rightarrow$ edge and stopping there at 85 ms. As it moves outward a communication segment appears and grows, $0\% \rightarrow 8.1\% \rightarrow 16.4\% \rightarrow 20.8\%$, which is precisely the cost being traded against the edge's better PUE. The stop is the noteworthy part: the core becomes reachable at 134 ms, yet AR never takes it, because the second transport hop would cost more than the $1.3 \rightarrow 1.2$ PUE improvement returns.

Taken together the three regimes confirm the veto interpretation and sharpen it. Latency removes tiers; energy chooses among what remains. A service is displaced by a tight budget only when the tier being removed is the one it would have chosen anyway — which is why the generative services pay about 30% for a tight deadline while the discriminative one pays nothing, and why neither the discriminative nor the AR service migrates to the core even when the deadline finally permits it.

IX. Conclusion and Future Work

This paper developed an end-to-end model of the energy use of multimodal AI inference across the core–edge–access compute continuum: computation, memory, and communication demands are derived analytically from model architecture and workload characteristics, every infrastructure resource is charged with a unified idle-plus-proportional power model including host, over-provisioning, and facility overheads, and energy-optimal placement, routing, and resource allocation are formulated as a MILP subject to end-to-end latency constraints. The analysis yields a closed-form placement rule: two infrastructure-dependent thresholds on the computation-to-communication ratio I partition the continuum, and the computation-to-memory ratio σ bends both thresholds inward for memory-bound services, which pay additional idle energy for the accelerators their state keeps powered.

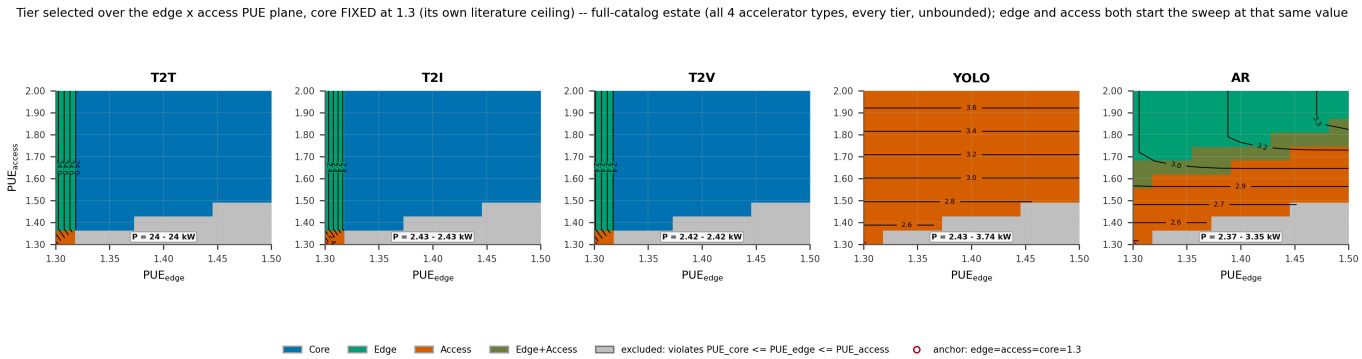


Fig. 12: Selected tier over the $\mathrm{PUE}_{\mathrm{edge}} \times \mathrm{PUE}_{\mathrm{access}}$ plane with the core pinned at $\mathrm{PUE}_{\mathrm{core}} = 1.3$, relaxed QoS. Colour is the tier the optimizer selects, black contours are total power, and the grey region is excluded because it would violate $\mathrm{PUE}_{\mathrm{core}} \leq \mathrm{PUE}_{\mathrm{edge}} \leq \mathrm{PUE}_{\mathrm{access}}$. The generative services are flat in both axes; YOLO depends on $\mathrm{PUE}_{\mathrm{access}}$ alone; AR is the only service whose placement responds to both.

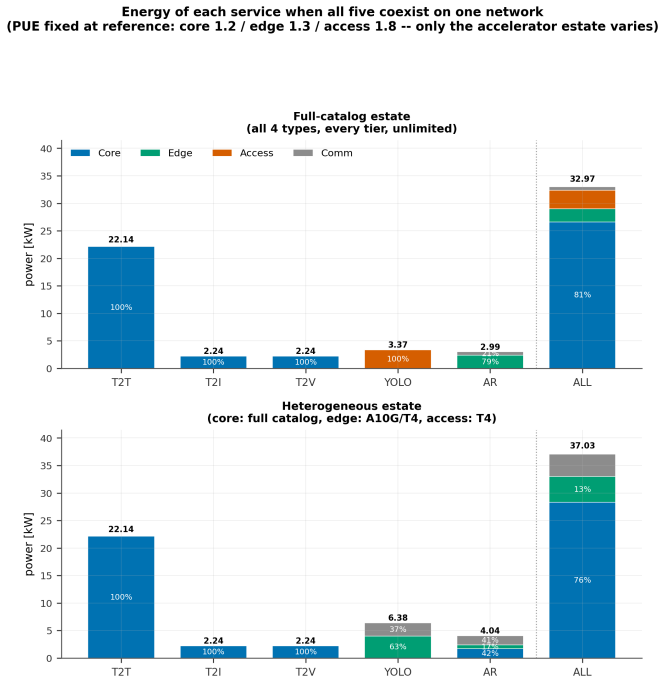


Fig. 13: Per-service energy when all five services share one estate, at the reference PUE cascade. Top: full accelerator catalogue at every tier. Bottom: the realistic estate (core full catalogue, edge A10G/T4, access T4 only). Bar height is absolute power and each segment is a tier's or communication's contribution; the rightmost bar is the estate total.

The evaluation validates the rule against the MILP optimum and traces its consequences. In the reference setting, computation-intensive generative services favor the core, video analytics the access tier, and augmented reality the edge. Placement migrates with the *ratio* of computation to communication efficiency gains: when computation efficiency improves faster, services drift toward users, and inward otherwise. Capacity improvements act asymmetrically: added memory cuts the accelerator count and power of memory-bound services, while added throughput at unchanged energy efficiency can raise power without reducing the count. Facility-efficiency scenarios show placement to be robust across admissible PUE orderings, with the services pinned at outer tiers most exposed to facility overhead. With delay budgets active, latency acts as a veto: it removes tiers from consideration, and energy chooses among the remainder, pushing interactive services outward even where pure energy favors the core.

Future work includes multi-function disaggregated services — prefill/decode separation and agentic pipelines, including conversational services that combine stateful autoregressive context with per-turn diffusion generation — as well as memory-bandwidth limitations and batching effects, and heterogeneous accelerator availability across tiers.

References

[1] International Energy Agency, "Energy and AI," International Energy Agency, Tech. Rep., 2025.

[2] Amazon Web Services, "AWS local zones," https://aws.amazon.com/about-aws/global-infrastructure/localzones/, 2026, accessed 2026.

[3] ETSI, "Multi-access edge computing (MEC); framework and reference architecture," ETSI, Tech. Rep. GS MEC 003, 2022.

[4] S. Rac and M. Brorsson, "Cost-aware service placement and scheduling in the edge-cloud continuum," *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 2, pp. 1–24, 2024.

[5] A. Shehabi, S. J. Smith, A. Hubbard, A. Newkirk, N. Lei, M. A. B. Siddik, B. Holecek, J. Koomey, E. Masanet, and D. Sartor, "2024 united states data center energy usage report," Lawrence Berkeley National Laboratory, Tech. Rep. LBNL-2001637, Dec. 2024.

[6] J. Llorca and A. M. Tulino, "Cloud network flow: Understanding information flow in next-generation cloud-integrated networks," *arXiv preprint*, 2025.

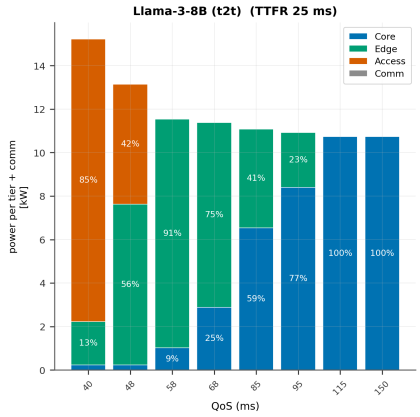
[7] A. Mauro, A. M. Tulino, and J. Llorca, "End-to-end orchestration of NextG media services over the distributed compute continuum," *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3068–3086, 2026.

[8] M. Barcelo, J. Llorca, A. M. Tulino, and N. Raman, "The cloud service distribution problem in distributed cloud networks," in *IEEE Int. Conf. Commun. (ICC)*, 2015, pp. 344–350.

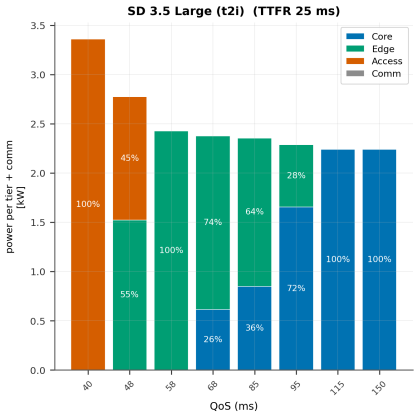
[9] M. Barcelo, A. Correa, J. Llorca, A. M. Tulino, J. L. Lopez, and A. Morell, "IoT-cloud service optimization in next generation smart environments," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 4077–4090, 2016.

[10] B. Addis, D. Belabed, M. Bouet, and S. Secci, "Virtual network functions placement and routing optimization," in *IEEE 4th Int. Conf. Cloud Netw.*, 2015, pp. 171–177.

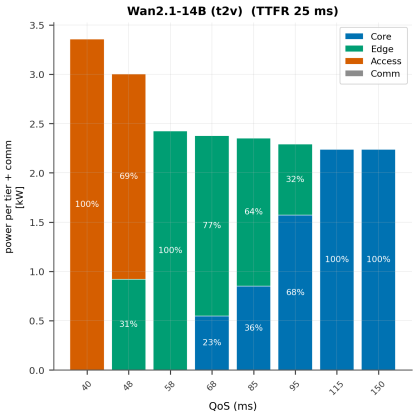
[11] P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warrier, N. Mahalingam, and R. Bianchini, "Characterizing power management opportunities for LLMs in the cloud," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.



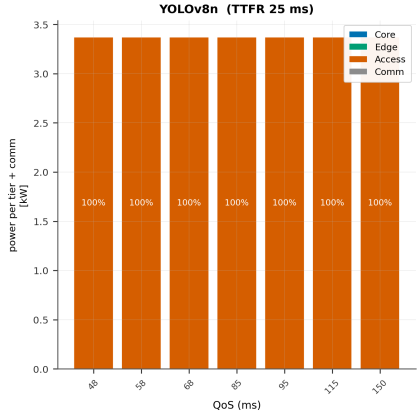
(a) T2T (Llama-3-8B).



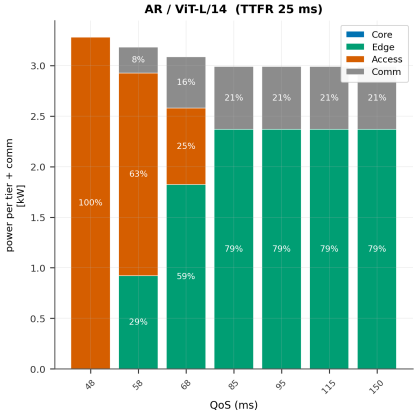
(b) T2I (SD 3.5 Large).



(c) T2V (Wan2.1-14B).



(d) YOLO (YOLOv8n).



(e) AR (ViT-L/14).

The generative services (a)–(c) migrate access→edge→core as the budget relaxes and save 29–33%; YOLO (d) is flat at the access tier over its whole feasible range; AR (e) stops at the edge and saves 8.9%, its communication segment growing as it moves outward.

Fig. 14: Per-tier energy composition versus the end-to-end QoS budget L_T^k. Bar height is total optimal power and each segment is that tier's (or communication's) share, labelled in-bar. Bars are absent where no placement is feasible.

[12] J. Stojkovic, E. Choukse, C. Zhang, I. Goiri, and J. Torrellas, "DynamoLLM: Designing LLM inference clusters for performance and energy efficiency," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.

[13] S. S. Vadari, "The model parking tax: Quantifying the hidden energy cost of always-on GPU model deployment," *arXiv preprint arXiv:2605.23918*, 2026, code and data: https://github.com/8bitai/gpu-parking-tax.

[14] Z. Yang, Y. Yang, C. Zhao, Q. Guo, W. He, and W. Ji, "PerLLM: Personalized inference scheduling with edge-cloud collaboration for diverse LLM services," *arXiv preprint arXiv:2405.14636*, 2024.

[15] A. Khoshsirat, G. Perin, and M. Rossi, "Decentralized LLM inference over edge networks with energy harvesting," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2024.

[16] L. Jin, Y. Zhang, Y. Li, S. Wang, H. H. Yang, J. Wu, and M. Zhang, "MoE^2: Optimizing collaborative inference for edge large language models," *arXiv preprint arXiv:2501.09410*, 2025.

[17] J. Baliga, R. W. A. Ayre, K. Hinton, and R. S. Tucker, "Green cloud computing: Balancing energy in processing, storage, and transport," *Proceedings of the IEEE*, vol. 99, no. 1, pp. 149–167, 2011.

[18] A. Vishwanath, F. Jalali, K. Hinton, T. Alpcan, R. W. A. Ayre, and R. S. Tucker, "Energy consumption comparison of interactive cloud-based and local applications," *IEEE Journal on Selected Areas in Communications*, 2015.

[19] G. Shen and R. S. Tucker, "Energy-minimized design for IP over WDM networks," *IEEE/OSA Journal of Optical Communications and Networking*, vol. 1, no. 1, pp. 176–186, 2009.

[20] W. Van Heddeghem, F. Idzikowski, W. Vereecken, D. Colle, M. Pickavet, and P. Demeester, "Power consumption modeling in optical multilayer networks," *Photonic Network Communications*, vol. 24, no. 2, pp. 86–102, 2012. [Online]. Available: https://backoffice.biblio.ugent.be/download/3234708/3234716

[21] S. F. Hernández, M. Pérez, and J. M. Rivas, "Power consumption modeling in multilayer IP-over-DWDM networks with coherent pluggables," *IEEE/OPTICA Journal of Optical Communications and Networking*, vol. 16, no. 4, pp. 340–352, 2024.

[22] "Information technology — data centres — key performance indicators — part 2: Power usage effectiveness (PUE)," International Organization for Standardization, Tech. Rep. ISO/IEC 30134-2:2016, 2016.

[23] Uptime Institute, "Global data center survey," Uptime Institute, Tech. Rep., 2025.

[24] X. Fan, W.-D. Weber, and L. A. Barroso, "Power provisioning for a warehouse-sized computer," in *Proc. ACM Int. Symp. Computer Architecture (ISCA)*, 2007.

[25] L. A. Barroso, U. Hölzle, and P. Ranganathan, *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, 3rd ed., ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2018.

[26] J. Chabarek, J. Sommers, P. Barford, N. Duffield, A. Greenberg, and R. Wright, "Power awareness in network design and routing," in *Proc. IEEE INFOCOM*, 2008.

[27] NVIDIA Corporation, "Data center GPU datasheets: T4, L4, L40S, A100, H100, B200," https://www.nvidia.com/en-us/data-center/, 2026, accessed Aug. 2026.

[28] A. Golden, C.-J. Wu, G.-Y. Wei, and D. Brooks, "The xPU-athalon: Quantifying the competition of AI acceleration," *arXiv preprint arXiv:2604.10852*, 2026.

[29] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated model-less inference serving," in *Proc. USENIX Annual Technical Conference (ATC)*, 2021.

[30] Z. Li, L. Zheng, Y. Zhong *et al.*, "AlpaServe: Statistical multiplexing

with model parallelism for deep learning serving," in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2023.
[31] A. Mishra, J. A. Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius, "Accelerating sparse deep neural networks," *arXiv preprint arXiv:2104.08378*, 2021.
[32] Optical Internetworking Forum, "Implementation agreement for 800ZR coherent optical interfaces," Optical Internetworking Forum, Tech. Rep. OIF-IA-800ZR-01.0, 2024.
[33] Juniper Networks, "400ZR and 400G OpenZR+ coherent optics guide," https://www.juniper.net/documentation/us/en/hardware/coherent-optics-cables-guide/, 2026, accessed Aug. 2026.
[34] Cisco Systems, "Cisco 8000 series routers data sheet," https://www.cisco.com/c/en/us/products/routers/8000-series-routers/index.html, 2026, accessed Aug. 2026.
[35] P. Mahadevan, P. Sharma, S. Banerjee, and P. Ranganathan, "A power benchmarking framework for network devices," in *Proc. IFIP Networking*, 2009.
[36] GÉANT Association, "The GÉANT pan-european network," https://network.geant.org/, 2026, accessed Aug. 2026.
[37] S. Nedevschi, L. Popa, G. Iannaccone, S. Ratnasamy, and D. Wetherall, "Reducing network energy consumption via sleeping and rate-adaptation," in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2008.
[38] ITU-T, "Characteristics of a single-mode optical fibre and cable," International Telecommunication Union, Tech. Rep. Recommendation G.652, 2016.
[39] R. A. K. Fezeu, E. Ramadan, W. Ye, B. Minneci, W. Xie, and Z.-L. Zhang, "An in-depth measurement analysis of 5G mmWave PHY latency and its impact on end-to-end delay," in *Proc. Passive and Active Measurement Conf. (PAM)*, 2023.
[40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
[41] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
[42] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
[43] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.
[44] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
[45] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 684–10 695.
[46] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.
[47] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
[48] J. Hoffmann, S. Borgeaud, A. Mensch *et al.*, "Training compute-optimal large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
[49] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
[50] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[51] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations (ICLR)*, 2021, arXiv:2010.02502.
[52] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.02747.
[53] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.
[54] Black Forest Labs, "FLUX.1: Open-weight rectified flow transformers for text-to-image generation," https://github.com/black-forest-labs/flux, 2024.
[55] P. Esser, S. Kulal, A. Blattmann *et al.*, "Scaling rectified flow transformers for high-resolution image synthesis," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
[56] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 684–10 695.
[57] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4195–4205.
[58] Wan Team, Alibaba, "Wan: Open and advanced large-scale video generative models," *arXiv preprint arXiv:2503.20314*, 2025.
[59] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
[60] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
[61] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.
[62] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
[63] J. D. C. Little, "A proof for the queuing formula: $l = \lambda w$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
[64] Ultralytics, "YOLOv8: Real-time object detection and image segmentation," https://github.com/ultralytics/ultralytics, 2023.
[65] L. Zheng *et al.*, "LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset," in *ICLR*, 2024, 1M conversations, 25 models, 210K users; mean 69.5 prompt / 214.5 response tokens (Llama-2 tokenizer), 2.0 turns.
[66] ISTAT, "Popolazione città metropolitana di milano," 2025.
[67] U. Nations, "World population prospects," 2024.
[68] 2025, openAI, ChatGPT usage figures (˜2.5 billion prompts/day), reported via Axios.
[69] E. Journal, "Ai image-generation volume estimates," 2024.
[70] 2025, estimated Sora video-generation volume, Forbes.
[71] 2025, kuaishou/Kling AI cumulative video generations.
[72] L. Zheng, W.-L. Chiang, Y. Sheng, T. Li, S. Zhuang, Z. Wu, Y. Zhuang, Z. Li, Z. Lin, E. P. Xing, J. E. Gonzalez, I. Stoica, and H. Zhang, "LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset," in *International Conference on Learning Representations (ICLR)*, 2024.

APPENDIX

A. Analytical Model Instantiation

To demonstrate the applicability of the analytical framework developed in Secs. V-B–V-D, we instantiate the proposed resource model using representative, openly documented foundation models covering the inference classes introduced in Sec. V-A. For each class, architectural parameters are obtained directly from the corresponding technical reports (Table VIII), representative inference workloads are derived from publicly available measurements (Table IX), and the resulting processing, memory, communication, and workload-factor values are computed by applying the analytical expressions developed in the previous sections.

a) Representative holding times and arrival rates: To quantify the aggregate resource demand (Table VI), the analytical framework requires the formulation of a per-class holding time, $\bar{\tau}$, and an arrival rate, λ. For the diffusion class, $\bar{\tau} = \mathrm{FLOPs}_{\mathrm{diff}}/(c^{\mathrm{flop}}u^{\mathrm{flop}})$, assuming an H100-class delivered throughput. This yields generation times of $\bar{\tau} \approx 5$ s for SD 3.5 Large (1024^2 resolution, 28 steps) and $\bar{\tau} \approx 220$ s for Wan2.1-14B (832×480 resolution, 81 frames), both of which align with empirical performance [58]. For the discriminative class, $\bar{\tau}$ corresponds to the per-frame service time. On a typical edge accelerator, this spans 5 to 15 ms, falling well below the 33 ms inter-frame arrival interval of a 30 fps video stream [64]; consequently, Little's Law applies in a zero-queueing regime. Conversely, the autoregressive class is stateful, meaning its holding time spans an entire multi-turn conversation. Assuming a decode-bound single-turn service time of approximately 5–10 ms/token for Llama-3-8B and 15–30 ms/token for Llama-3-70B [11], [49], a standard response of $T_{\mathrm{resp}} = 214$ tokens requires $\tau_{\mathrm{req}} \approx$ 2-5 s. When aggregated over an average session of $N_t \approx 2$ turns (LMSYS-Chat-1M [65]) that incorporates user think times, the total conversation holding time, $\bar{\tau}_{\mathrm{conv}}$, extends to the order of one minute.

Arrival rates are calibrated against published global usage statistics. Although the energy-optimal placement of a service does not depend on its offered load (Sec. VII-B), we formulate absolute rates for a representative urban center (the Metropolitan City of Milan, $P_{\mathrm{city}} \approx 3.2 \times 10^6$ [66]) to ground the numerical analysis. The per-class arrival rate is scaled proportionally via $\lambda_c = (V_c^{\mathrm{glob}}/P_{\mathrm{world}})P_{\mathrm{city}}/86400$, assuming a global population of $P_{\mathrm{world}} \approx 8.2 \times 10^9$ [67]. Based on reported daily volumes of $V_{\mathrm{T2T}}^{\mathrm{glob}} \approx 2.5 \times 10^9$ prompts/day [68], $V_{\mathrm{T2I}}^{\mathrm{glob}} \approx 3.4 \times 10^7$ images/day [69] (peaking near 10^8/day at the GPT-4o image launch), and $V_{\mathrm{T2V}}^{\mathrm{glob}} \approx 10^7$ videos/day [70], [71], we derive a generalized usage funnel of $\mathrm{T2T} : \mathrm{T2I} : \mathrm{T2V} \approx 1 : 1.4 \times 10^{-2} : 4 \times 10^{-3}$. This yields local request rates of $\lambda_{\mathrm{T2T}} \approx 11.3$, $\lambda_{\mathrm{T2I}} \approx 0.15$, and $\lambda_{\mathrm{T2V}} \approx 0.045$ requests/s. For autoregressive tasks, the state-holding unit is the conversation; hence, the effective rate becomes $\lambda_{\mathrm{conv}} = \lambda_{\mathrm{req}}/N_t \approx 5.65$ conv/s.

In the experiments of Sec. VIII, these rates are scaled up (Table V), which by load invariance changes absolute power but not placement.

b) Representative inference workloads: The analytical model requires a small set of workload descriptors that determine the per-request resource demand of each inference class. These descriptors are selected from publicly available datasets and model documentation to reflect representative operating conditions.

For the autoregressive class, the average prompt and response lengths, $T_{\mathrm{req}} = 69.5$ and $T_{\mathrm{resp}} = 214$ tokens, are obtained from the LMSYS-Chat-1M corpus, which contains one million real user conversations [72]. For diffusion models, the denoising configuration (number of inference steps and classifier-free guidance factor) and rendering resolution correspond to the default inference settings reported for the respective models [55], [58]. The resulting latent-token counts are computed from Eqs. (16)–(17), yielding $T_{\mathrm{vis}} = 4096$ for 1024×1024 image generation and $T_{\mathrm{vis}} = 32760$ for an 832×480, 81-frame video. Finally, the discriminative workload is represented by a 1080p, 30-fps H.264 video stream, whose bitrate determines the communication demand. For the two audio services we take Whisper-large-v3 (encoder 0.64 B / decoder 0.91 B parameters) processing 30 s clips ($N_{\mathrm{in}} = 1500$ encoder tokens, $T_{\mathrm{resp}} = 100$ output tokens), and Moshi (7.7 B parameters) at an codec frame rate $f_{fr} = 12.5$ frames/s with $K = 8$ codebooks ($\phi_{aux} \approx 0.2B$) over 5+5 s exchanges with a conversation-level holding time $\bar{\tau} = 30$ s. Audio is transmitted as lossless 16 kHz 16-bit PCM (256 kbps). With these descriptors both services fall in the intermediate band $I = F/B \in [10^5, 10^7]$ FLOP/bit, between the communication-bound stream analytics and the compute-bound generative models.

c) Model instantiation: Substituting the architectural parameters of Table VIII and the workload descriptors of Table IX into the analytical models of Secs. V-B–V-D yields the per-request resource requirements reported in Table V.

TABLE VIII: Representative foundation models and architectural parameters.

I/O Modality	Model	Paradigm	Φ_{dn}[B]	Φ_{tot}[B]	Architecture
T2T	Llama-3-8B	Autoregressive	8.03	8.03	L=32, d=4096, n_kv=8, d_head=128
T2I	SD 3.5 Large	Diffusion	8	13.7	L=37, d=2432, image
T2V	Wan2.1-14B	Diffusion	14	19.6	L=40, d=5120, video
VID	YOLOv8n	Discriminative	0.0032	0.0032	8.7GFLOP/frame
AR	AR / ViT-L/14	Discriminative	0.304	0.304	80.7GFLOP/frame
S2T	Whisper-v3	Autoregressive	1.54	1.54	enc-dec Transformer; audio enc + AR text dec
A2A	Moshi	Autoregressive	7.69	7.69	L=32, d=4096, n_kv=8, d_head=128; audio tokens

TABLE IX: Representative inference workloads (per request).

I/O Modality	Model	T_{req}	T_{resp}	Resolution	$N_{\mathrm{steps}} \cdot g$	T_{vis}	stream
T2T	Llama-3-8B	69.5	214	-	-	-	-
T2I	SD 3.5 Large	38	-	1024x1024	28*2	4.1×10^3	-
T2V	Wan2.1-14B	50	-	480x832,81f	50*2	3.28×10^4	-
VID	YOLOv8n	-	-	1920x1080@30fps	1 pass	-	3.7Mbps (H.264)
AR	AR / ViT-L/14	-	-	1920x1080@30fps	1 pass	-	19Mbps (intra-LL)
S2T	Whisper-v3	1500	100	-	-	-	256kbps PCM, 30s
A2A	Moshi	62.5	62.5	-	-	-	256kbps PCM, 5+5s

TABLE X: Decomposition of representative foundation models into the structural components of an inference function ($\mathcal{J} = \mathcal{J}_{\mathrm{enc}} \cup \{\mathrm{bb}\} \cup \mathcal{J}_{\mathrm{aux}} \cup \mathcal{J}_{\mathrm{dec}}$).

Model	**Encoders** ($\mathcal{J}_{\mathrm{enc}}$)	**Backbone** (bb)	**Auxiliary** ($\mathcal{J}_{\mathrm{aux}}$)	**Decoder** ($\mathcal{J}_{\mathrm{dec}}$)
Llama-3	—	decoder-only Transformer	—	—
Multimodal LLM	vision encoder (ViT)	LLM Transformer	—	—
Whisper	audio Transformer encoder	text decoder Transformer	—	—
SD 3.5 Large	CLIP-L, CLIP-G, T5-XXL	MM-DiT	—	VAE decoder
Wan2.1	umT5	cross-attention DiT	—	Wan-VAE decoder
SD 1.5	CLIP	U-Net (CNN)	—	VAE decoder
Moshi	Mimi encoder	temporal Transformer (Helium)	depth Transformer	Mimi decoder
YOLOv8	—	whole network	—	—
ViT-L/14 (AR)	—	whole ViT	—	—